METROPOLITAN STATE UNIVERSITY

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

FINAL YEAR PROJECT REPORT

ANIHUB

STUDENTS NAMES:

Jimmie Xiong

May 2025

“I hereby declare that I have read this project report and in my opinion this report is sufficient in terms of scope and quality for the award of the degree of Bachelor in Computer Science

Signature : .....

Supervisor :

Date :

FINAL YEAR PROJECT REPORT

ANIHUB

STUDENTS NAMES:

Jimmie Xiong

A report submitted in partial fulfillment of the requirements for the award of the Bachelor of Science in "Computer Science and Computer Information Technology" Major taken

College of Science,

Computer Science and Cybersecurity

May 2025

DECLARATION

I declare that this thesis entitled “AniHub” is the result of my/our own research except as cited in the references. The report has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Signature : .....

Name : Jimmie Xiong

Date : 05/05/2025

ACKNOWLEDGEMENT

I am proud to present *AniHub*, a capstone project that I conceptualized, designed, and developed independently. This endeavor allowed me to apply and expand my skills in software development, problem-solving, and project management. Throughout the project, I integrated and adapted various external APIs to enhance functionality, ensuring that all integrations were customized to align with the project's objectives.

This project stands as a testament to my dedication and commitment to personal and professional growth in the field of computer science.

ABSTRACT

AniHub is a full-stack anime and donghua streaming platform built to streamline content discovery and elevate the user experience by consolidating fragmented sources into a single, modern interface. Developed as a solo capstone project, AniHub allows users to browse curated anime and donghua, filter by genre and origin (Japanese or Chinese), manage a personal watchlist, and securely stream episodes from third-party sources like AniWatchTV.

The application leverages web scraping, decryption, and dynamic content loading to fetch real-time anime data, episodes, and streaming sources. The frontend is built with React and styled-components to deliver a responsive UI, while the backend uses Node.js and TypeScript to structure modular API routes for metadata, search, and streaming logic.

User management is handled through Firebase Authentication, with Firestore used for storing watchlists, subscription status, and account information. The app includes route protection to distinguish between guest, registered, and premium users, restricting access to features like My List and streaming content based on subscription level. Stripe integration powers the premium payment system, and the backend securely updates user records upon successful checkout using the Firebase Admin SDK.

AniHub reflects not only strong full-stack engineering and real-world application of web technologies, but also deep research into how popular platforms and similar services structure content delivery and user flow. This project successfully demonstrates the integration of front-end design, backend services, user authentication, payment systems, and secure content delivery all tailored to solve real user pain points in the anime streaming ecosystem.

Table of Contents

INTRODUCTION 9
2.0 VISION AND BUSINESS CASE AND SCOPE 9
2.1 VISION 10
2.2 BUSINESS CASE 10
2.3 SCOPE 11
3.0 FEASIBILITY STUDIES 11
3.1 The system Functionalities 11
3.2 Proposed system functionalities 12
3.4 Problem Statement 12
3.5 Goals 13
3.5 Requirement Study 13
4.0 DETAIL DESIGN 13
4.1 Gathered requirements 14
4.2 Prototype 15
5.0 SYSTEM DESIGN 19
5.1 SYSTEM 19
5.1.1 USE CASE DIAGRAM 19
5.1.2 SYSTEM USERS 19
6.0 SEQUENCE DIAGRAM 20
6.0.1 SYSTEM UNREGISTERED USER FLOW 20
6.1.2 SYSTEM REGISTERED USER (NON-PREMIUM) 21
6.1.3 SYSTEM PREMIUM USER 24
7.0 INTERFACE DESIGN 32
8.0 DATABASE 33
9.0 CLASS DIAGRAM 34
10.0 PROJECT SYSTEM 34
10.1.1 SYSTEM FUNCTIONALITY 34
11.0 SYSTEM REQUIREMENT 36
11.1.1 SOFTWARE REQUIREMENT 36
11.1.2 HARDWARE REQUIREMENT 36
12.0 SYSTEM IMPLEMENTATION 37
12.1.1 SUBSCRIPTION PLAN SELECTION PAGE 37
12.1.2 AUTHENTICATED HOME DASHBOARD 38
12.1.3 LATEST EPISODE UPDATE PAGE 39
12.1.4 PREMIUM MOVIE PAGE 40

12.1.5 ANIME CARD INTERACTION 40
12.1.6 MY LIST PAGE 41
12.1.7 USER PROFILE MANAGEMENT 42
12.1.8 ACCOUNT SETTING & CREDENTIAL UPDATE 42
12.1.9 ANIME DETAIL PAGE 43
12.2.0 EPISODE PLAYBACK PAGE 44
13.0 SYSTEM MODIFICATION 45
13.1.1 SYSTEM USER MODULE 45
14.0 SYSTEM LIMITATION 45
15.0 SYSTEM FUTURE ENHANCEMENT 46
16.0 PROJECT SCHEDULING 46
16.1.1 SCHEDULING TASKS DURATION 46
16.1.2 PROJECT TIMELINE 47
17.0 CONCLUSION 48
18.0 REFERENCES 49

LIST OF TABLES

NO	TABLES	DESCRIPTION
1	Table 1.0	Tasks duration

LIST OF FIGURES

NO	FIGURES	DESCRIPTION
1	Figure 1	Login Prototype Page
2	Figure 2	Create Account Prototype Page
3	Figure 3	AniHub Home Page
4	Figure 4	My List Prototype
5	Figure 5	Anime Card Component (Hover Interaction)
6	Figure 6	Use Case Diagram for AniHub
7	Figure 7	Unregistered User Signup and Access Control
8	Figure 8	Registered User Login
9	Figure 9	Non-Premium User Subscription Upgrade
10	Figure 10	Registered User Logout
11	Figure 11	User Profile and Account Management
12	Figure 12	Premium User Homepage
13	Figure 13	Premium User Origin Filter and Pagination
14	Figure 14	Premium User Viewing Latest Episodes
15	Figure 15	Premium User Viewing Movies
16	Figure 16	Premium User Accessing “My List”
17	Figure 17	Premium User Watching an Episode
18	Figure 18	Premium User Performing a Search
19	Figure 19	Login Interface
20	Figure 20	Create Account Interface
21	Figure 21	Firebase Firestore Database Schema for AniHub
22	Figure 22	Premium Subscription Selection Page
23	Figure 23	Authenticated Home Dashboard

24	Figure 24	Latest Episodes Page
25	Figure 25	Premium Movies Page
26	Figure 26	Anime Card Interface
27	Figure 27	My List Page
28	Figure 28	User Profile Management Interface
29	Figure 29	Account Settings and Credential Update
30	Figure 30	Anime Detail Page
31	Figure 31	Episode Playback Page

INTRODUCTION

AniHub is a full-stack anime and donghua streaming web application developed as a solo capstone project by Jimmie Xiong, a Bachelor of Science in Computer Science student at Metropolitan State University, under the mentorship of Dr. Ismail Bile Hassan. The primary objective of this project was to create a centralized platform that streamlines the discovery, organization, and playback of animated content specifically targeting Japanese anime and Chinese donghua enthusiasts.

Modern anime viewers often face fragmented access, unreliable streaming sources, and inconsistent user interfaces across platforms. AniHub addresses these challenges by aggregating content from third-party providers such as AniWatchTV into a single, user-friendly interface that mimics the functionality and design philosophy of commercial platforms.

The frontend was developed using React and styled-components to ensure a responsive, modern interface. The backend, powered by Node.js and TypeScript, includes modular APIs for delivering dynamic data such as anime metadata, episode lists, search results, and streaming URLs. Web scraping and custom decryption logic were implemented using Axios, Cheerio, and proprietary handlers to fetch playable links from sources like MegaCloud, while gracefully handling CORS restrictions.

User authentication and role-based access control are managed via Firebase Authentication. Firestore serves as the NoSQL database for storing user-specific data such as watchlists, profile settings, and premium

subscription status. Premium features such as full episode streaming, the Movies page, and My List are gated behind protected routes, with payment processing handled securely through Stripe integration. Upon successful checkout, the backend updates the user's premium status using Firebase Admin SDK, unlocking access in real time.

AniHub was built and tested entirely in a local development environment using Visual Studio Code, with a clear separation between frontend and backend layers. The project showcases the developer's proficiency in full-stack development, API design, user authentication, payment integration, and frontend engineering. More importantly, it reflects a genuine passion for solving real-world user pain points in the anime streaming ecosystem by combining modern web technologies with research into how established streaming sites operate behind the scenes.

2.0 VISION AND BUSINESS CASE AND SCOPE

2.1 VISION

In today's digital age, anime content is spread across multiple platforms, making it difficult for viewers to find, track, and stream their favorite shows in one place. This fragmentation affects both casual viewers and dedicated fans who want a seamless and organized viewing experience.

The vision of AniHub is to provide a centralized, user-friendly streaming platform that simplifies anime and donghua discovery, streaming, and management. By aggregating high-quality content from sources and offering smart features such as personalized watchlists, genre filtering, and origin-based sorting, AniHub aims to enhance how users interact with animated content.

Additionally, AniHub envisions a platform that respects user security and access control, offering premium features through secure authentication and integrated subscription payments. Whether users are long-time anime fans or new to the world of donghua, AniHub strives to be their all-in-one hub for immersive and enjoyable streaming.

2.2 BUSINESS CASE

In the current streaming landscape, anime and donghua content is scattered across numerous platforms, many of which are inconsistent, ad-heavy, untrusted, or unreliable. Users often struggle with broken links, limited search functionality, or region-locked content. This creates frustration and drives users to constantly switch between sites or rely on unofficial sources.

AniHub addresses this gap by offering a centralized platform that aggregates streaming sources into a single, streamlined experience. By simplifying the discovery, filtering, and playback of anime and donghua content, the platform creates a practical solution for fans who want reliable access without the hassle.

From a business perspective, AniHub demonstrates how full-stack development, web scraping, and API integration can be used to build a real-world solution for media aggregation. Furthermore, its Stripe-backed subscription system introduces a monetization pathway that supports scalable growth through premium features—showcasing the potential for future business models in niche streaming markets.

2.3 SCOPE

The purpose of this application is to create a centralized streaming platform for users who are interested in discovering, watching, and managing anime and donghua content more efficiently. Viewers often struggle with fragmented access, inconsistent quality, and lack of organization across various anime sources.

This project aims to optimize that experience by consolidating data from trusted third-party sources into a single platform. It includes features such as anime browsing, episode playback, filtering by genre or origin, and managing user-specific watchlists. The system also supports premium content access through Stripe-powered subscriptions and Firebase-authenticated user accounts.

AniHub is developed as a local full-stack web application with separate frontend and backend environments. The platform is intended for personal and educational use, showcasing a fully functional model for future streaming platforms targeting niche audiences.

3.0 FEASIBILITY STUDIES

3.1 The system Functionalities

The feasibility of the AniHub project was assessed in two major areas: technical feasibility and economic feasibility.

Technical Feasibility: The project was developed using widely supported and reliable technologies such as React, Node.js, TypeScript, Firebase, and Stripe. These tools are fully compatible with standard development environments and require no specialized hardware or infrastructure. All development and testing were performed locally using Visual Studio Code, with a modular separation between frontend and backend. The use of Firebase Authentication and Firestore ensures secure and scalable data handling, while Axios and Cheerio support dynamic scraping and content aggregation. Given the maturity and availability of these technologies, the system is technically feasible and sustainable for future improvements.

Economical Feasibility: AniHub demonstrates strong economic feasibility through its low development cost and high functional return. The platform was developed independently using open-source libraries and tools, avoiding licensing fees or infrastructure costs. Its local deployment model reduces overhead while still showcasing a fully working subscription-based streaming service. By streamlining access to anime and donghua content, AniHub saves users time and provides a smoother viewing experience. In the long term, the subscription system powered by Stripe presents an opportunity for monetization, demonstrating potential for growth and user value beyond the scope of the academic project. The benefits clearly outweigh the development costs, and the system passes the feasibility study on both technical and economic grounds.

3.2 Proposed system functionalities

The proposed AniHub system is designed to eliminate the fragmented experience users face when trying to discover and stream anime and donghua across multiple unreliable websites. By creating a centralized web application, users can access high-quality content, organized by genre, origin, popularity, latest updates shows, and movies all from one responsive interface.

The system allows users to:

- Search and stream anime episodes through embedded sources like MegaCloud
- View complete information for each series, including synopsis, and episode count
- Filter anime by type (TV, Movie) and origin (Japan or China)
- Maintain personal watchlists
- Securely sign in using Firebase Authentication
- Subscribe for premium content through Stripe and unlock premium content

This system enhances user experience, improves access reliability, and reduces the need to jump between platforms. It is fully responsive and can be accessed from any device with a web browser.

3.4 Problem Statement

The current landscape of anime and donghua streaming is plagued by unreliable, ad-heavy websites, broken links, limited search functionality, and a lack of personalization. Users often waste time searching across multiple platforms or using unofficial sites with inconsistent quality and security risks.

One major problem is the lack of a unified platform that combines streaming, discovery, and tracking features. Additionally, there is no smooth system in place to gate premium content or reward creators with a subscription-based model in this niche.

AniHub was created to solve these problems by aggregating content into a single, user-friendly platform with powerful search and filter features, integrated video playback, and optional premium access. The system is designed to streamline the anime-watching experience while demonstrating technical and business scalability.

3.5 Goals

The goal of this project is to develop a centralized anime and donghua streaming system that addresses the issues highlighted in the problem statement. The platform aims to streamline content discovery, simplify streaming access, and provide users with a smooth and modern interface for managing and watching their favorite shows.

To achieve this, the project first involved identifying what users need in an anime platform such as reliable video playback, easy browsing, clean organization, and personalized access. A prototype was created to visualize the solution, tested for usability, and refined based on functionality and user flow. The final product was implemented as a responsive web application that runs locally and can be accessed from any modern browser. It integrates features like episode streaming, search, watchlist management, origin filtering, and premium subscription handling.

3.5 Requirement Study

The goal of the requirements study was to determine how the system should be designed to meet the expectations of modern anime and donghua viewers. This included researching how users typically discover and consume anime content, and what features they find most valuable such as genre filters, playback speed, clean UI, and mobile-friendly layouts. The study involved analyzing existing anime streaming platforms, identifying common frustrations (like broken links or missing episodes), and finding ways to improve the user experience. Another goal was to explore technical requirements such as scraping logic, secure authentication, and Stripe-based premium access, ensuring the system supports both casual viewers and advanced users who want more control over their viewing experience.

4.0 DETAIL DESIGN

This section describes the detailed system design and activity flows for the AniHub anime and donghua streaming platform. It outlines how users interact with the application and how each component frontend, backend, Firebase, and Stripe communicate to deliver a smooth and secure streaming experience.

The system is divided into multiple activity flows, each representing a major feature of the platform:

- **User Authentication Flow**

- Users sign up or log in using Firebase Authentication. The app verifies credentials and grants access to protected routes based on the user's login state.

- **Anime Discovery and Streaming Flow**
  - Users can browse, search, or filter anime by genre, type, or origin. Selecting a show triggers a backend request to fetch details and episodes. When an episode is selected, the app fetches video sources (MegaCloud) and begins playback.

- **Watchlist and User Data Flow**
  - Logged-in users can save anime to their personal watchlist. This data is stored in Firebase Firestore and retrieved each time the user visits their list.
- **Premium Subscription Flow**
  - Users choosing to upgrade are redirected to a Stripe Checkout page. Upon successful payment, the backend uses the Firebase Admin SDK to mark the user as premium. Premium-only content is gated using protected route logic.

Each of these processes is represented by activity diagrams that show the steps users take, the API endpoints called, and the interactions between the frontend, backend, and external services.

4.1 Gathered requirements

Some of the key requirements gathered for the AniHub anime and donghua streaming platform are outlined below. These requirements ensure the platform delivers a smooth, responsive, and personalized user experience.

The system must have:

- **Support for user authentication and protected content access**
  - Users must be able to register, log in, and access personalized features such as watchlists and premium-only content.
- **Functionality to browse, search, and filter anime**
  - Users must be able to search anime titles, browse by genre, filter by origin (Japanese or Chinese), and view categorized sections like trending and most popular.
- **Streaming support with episode-level playback**
  - The system must allow users to click on an anime, view its episodes, and manually stream selected episodes through a third party player like VLC media player by copying the decrypted MegaCloud source.
- **Watchlist**

- Users should be able to save anime to a “My List” page. This list must persist across sessions and allow users to track their favorite titles.
- **Premium subscription management**
  - The system must allow users to upgrade to premium via Stripe. Upon successful payment, premium content becomes accessible through protected routes.
- **Display of detailed anime information**

  Each anime must show a title, description, genres, type, episode count, release year, and origin.
- **Responsive and device-friendly interface**
  - The system must be accessible from any device with a browser and adjust layout accordingly for mobile, tablet, or desktop use.
  -

4.2 Prototype

The AniHub prototype was created to visually and functionally demonstrate the core features and flow of the application. The design emphasizes simplicity, responsiveness, and a streaming experience inspired by Netflix. Below are the key screens from the prototype:

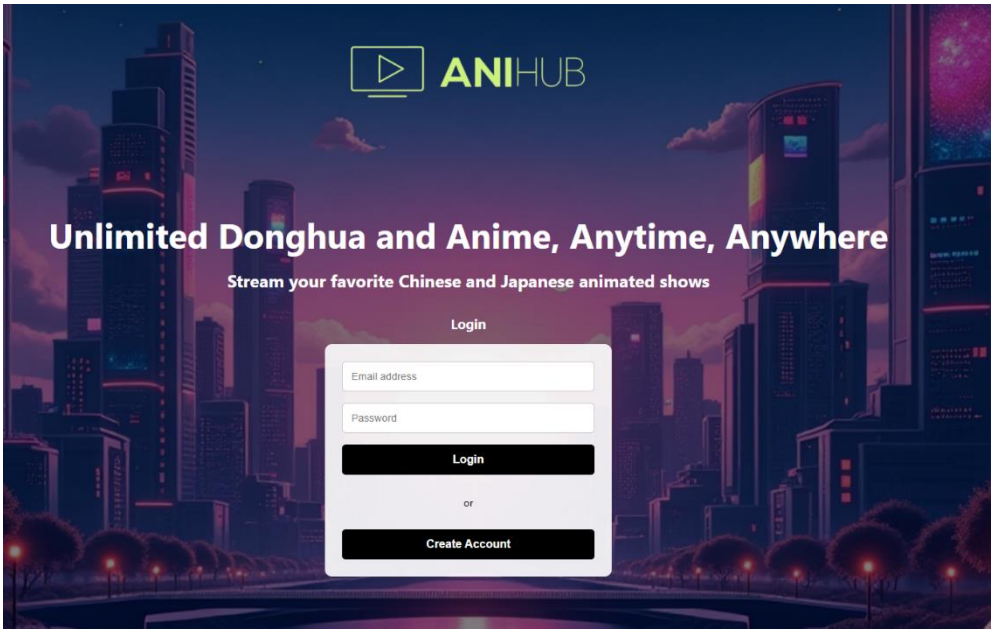

Figure 1: Login Prototype Page

Figure 1 illustrates the login screen prototype, which serves as the initial user interface of AniHub. Both new and returning users can either sign in or create an account through this screen. It features a clean, modern design with a prominent tagline emphasizing the platform’s focus on streaming Chinese and Japanese animated

content. The interface includes input fields for email and password, along with clearly labeled buttons for login and account creation, acting as the gateway into AniHub's authentication and access flow.

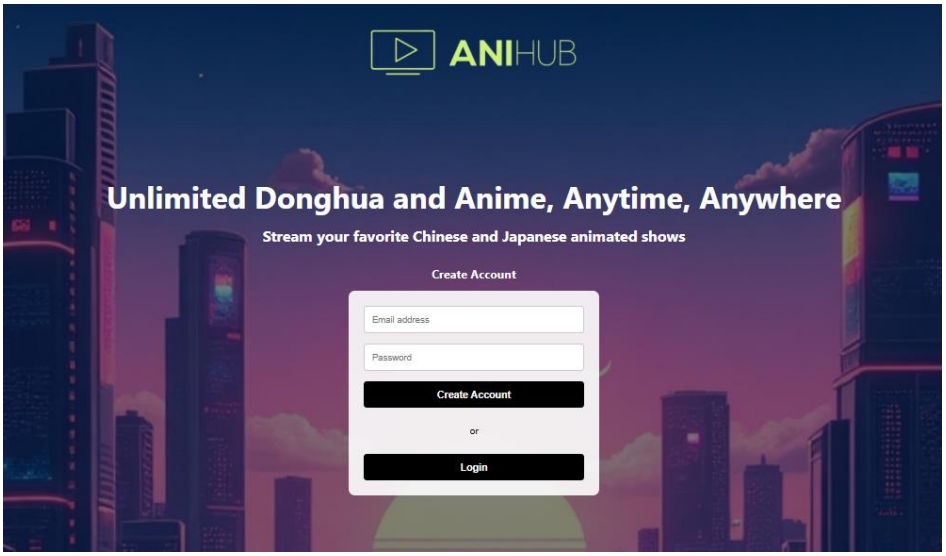

Figure 2: Create Account Prototype Page

Figure 2 illustrates the Create Account screen prototype of AniHub, designed for unregistered users to register a new account. It closely mirrors the login screen, with the same layout and design, but the positions of the "Login" and "Create Account" buttons are reversed.

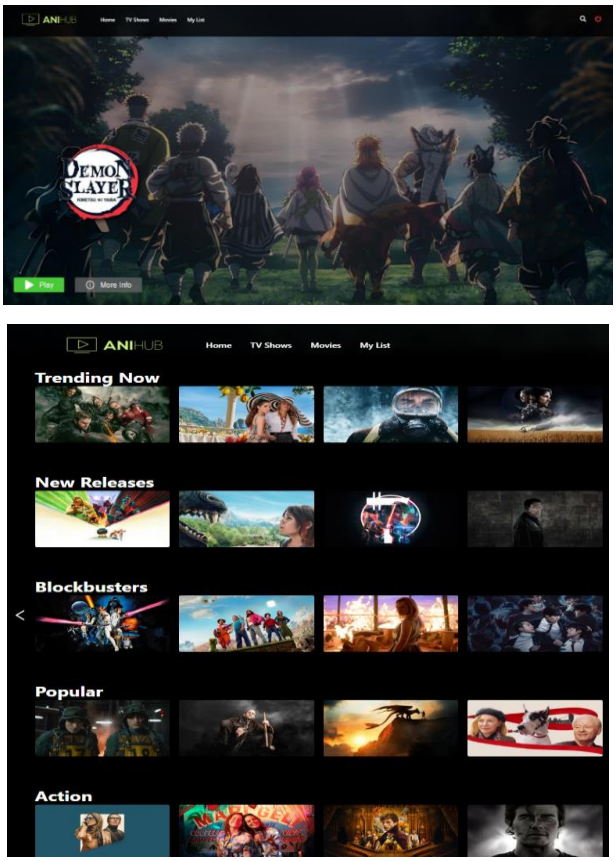

Figure 3: Home Page

Figure 3 showcases the prototype for AniHub Home Page, which serves as the central hub for exploring and streaming content. The top section features a full screen banner with alongside quick-access buttons for "Play" and "More Info." Below the banner, users can scroll through categorized sections such as Trending Now, New Releases, Blockbusters, Popular, and Action. Each category presents a horizontal slider of curated anime and animated titles, offering an engaging and visually rich browsing.

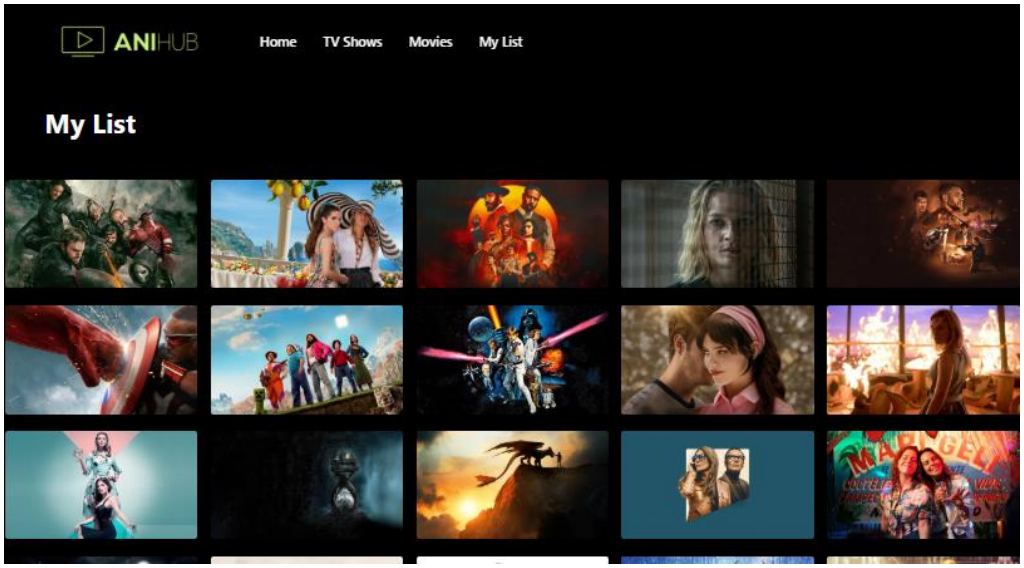

Figure 4: My List Prototype

Figure 4 displays the prototype My List page of AniHub, which allows users to view and manage the anime titles they've personally saved. This section provides a personalized watchlist, displaying all content the user has added through the "Add to My List" feature across the platform. Each anime is visually represented by a thumbnail, arranged in a clean, scrollable grid layout. This feature enhances the user experience by making it easy to return to favorite or unfinished shows without searching again.

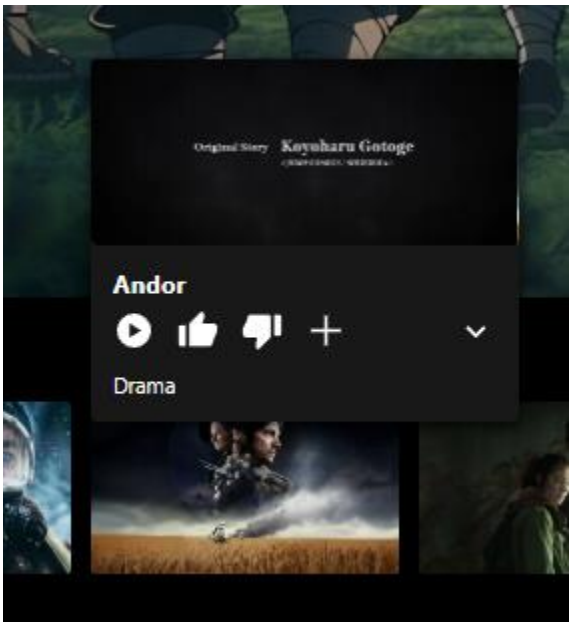

Figure 5: Card Prototype

Figure 5 shows the interactive card component used to display anime or show previews across AniHub. When a user hovers over a thumbnail, the card expands to reveal additional details such as the title, genre, and action buttons. Users can instantly play the content, like or dislike it, or add it to My List by clicking the plus sign for later viewing.

5.0 SYSTEM DESIGN

5.1 SYSTEM

5.1.1 USE CASE DIAGRAM

5.1.2 SYSTEM USERS

The AniHub streaming platform is designed to provide a seamless experience for users who wish to stream Chinese and Japanese animated content all in one. The system supports multiple user roles Unregistered Users, Registered Users, and Premium Users each with different levels of access and functionality.

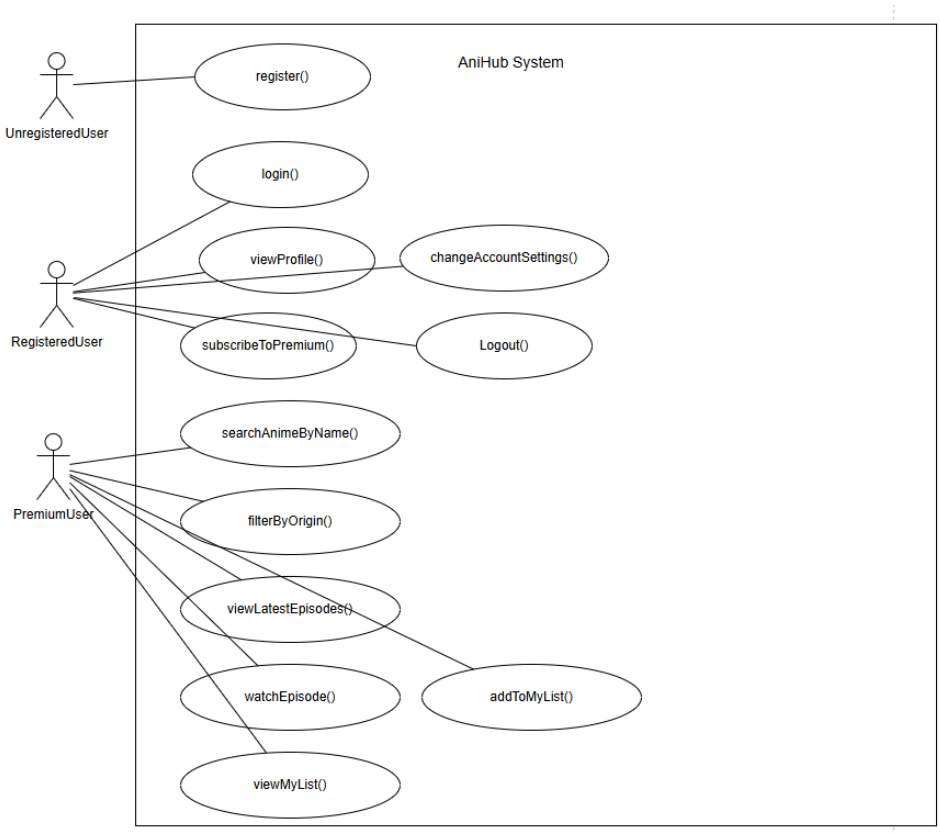

Figure 6: Use Case Diagram

Figure 6 Illustrates AniHub system's use case diagram

There are three main actors:

- **Unregistered User**
  - Has access to minimal functionality. Can only register (sign up).
- **Registered User**
  - Once signed in, can manage their account settings, subscribe to a premium plan, and view their profile. They serve as the base role, unlocking core user functionality.
- **Premium User**

- Inherits all permissions from the Registered User. Additionally, they gain full access to exclusive features such as searching anime by name, filtering anime by origin, viewing the latest episodes, watching specific episodes, and managing a personalized “My List”.

6.0 SEQUENCE DIAGRAM

6.0.1 SYSTEM UNREGISTERED USER FLOW

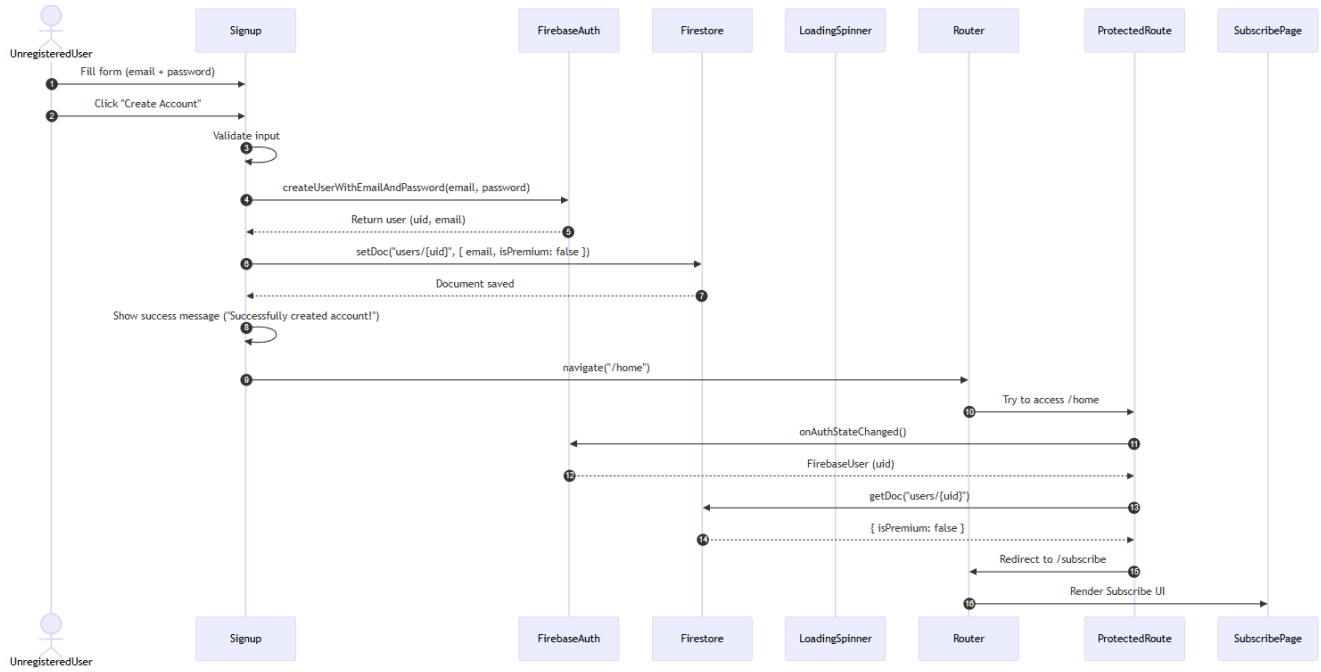

Figure 7: Unregistered User Signup and Access Control

Figure 7 illustrates the signup flow for unregistered users on AniHub. When a user navigates to the signup page, the system ensures they are not already authenticated before granting access to the form. After entering their email and password and clicking “Create Account,” the app uses Firebase Authentication to register the user. A corresponding user document is then created in Firestore with their email and a default isPremium: false status. Once account creation is successful, the user will be taken to the home page, but a loading screen is displayed preventing them from seeing the homepage, behind this the system checks their premium status. Since the user is not yet premium, they are redirected to the subscribe page to begin the upgrade process.

6.1.2 SYSTEM REGISTERED USER (NON-PREMIUM)

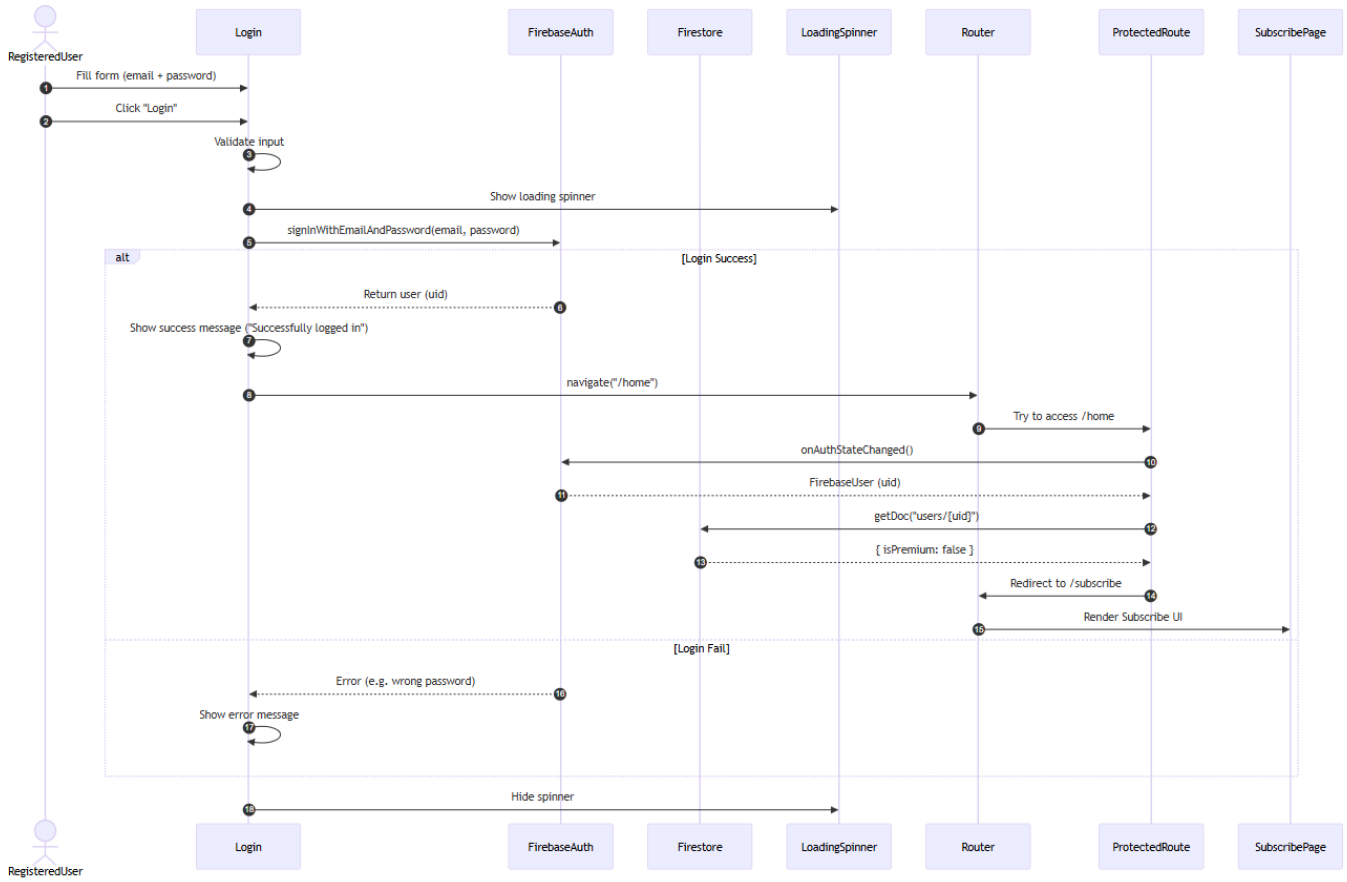

Figure 8: Login Sequence

Figure 8 illustrates the login and access flow for registered AniHub users. When a user submits their email and password, the system uses Firebase Authentication to verify the credentials. Upon successful login, the user is redirected to the home route. The system then checks the user's premium status in Firestore. If the user is not a premium member, they are redirected to the subscribe page. If authentication fails, an error message is shown, and the loading spinner is removed.

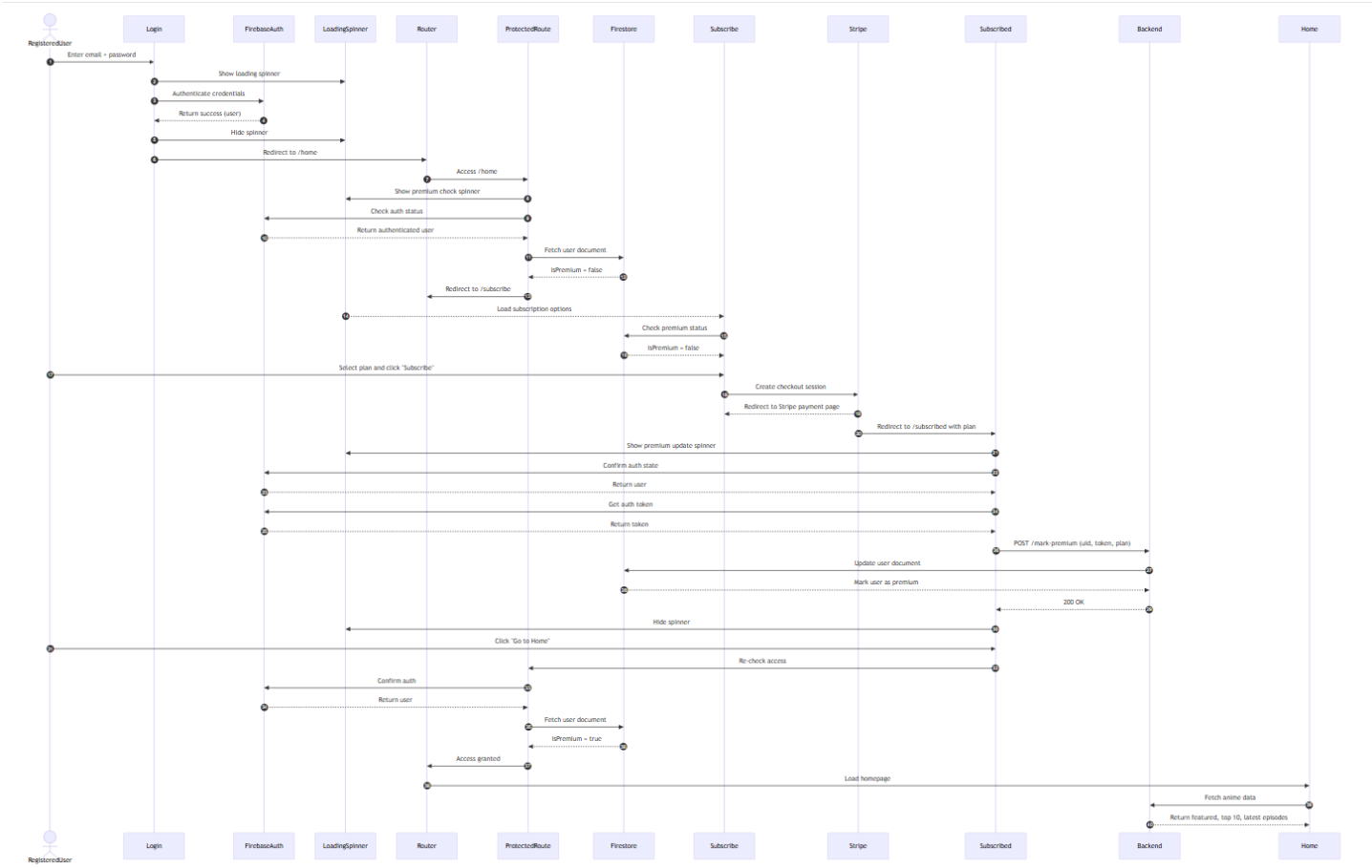

Figure 9: Registered Non-Premium User Subscription Upgrade

Figure 9 illustrates the complete user journey of a registered non-premium AniHub user upgrading to premium status. Upon logging in with valid credentials, the system authenticates the user through Firebase Authentication while a loading spinner provides visual feedback. The user is then redirected to the home route, where the ProtectedRoute component checks their premium status in Firestore. If the user is not yet a premium member, they are redirected to the subscription page. There, the user selects a subscription plan and is redirected to Stripe Checkout. Upon successful payment, Stripe returns the user to the subscribed route, where their premium status is updated in Firestore via a secure backend request authenticated with a Firebase ID token. During this update, a loading spinner is shown to indicate progress. Once complete, the user is welcomed to AniHub Premium and redirected to the home page. The system rechecks the user's premium status and grants full access to premium content. This sequence ensures secure authentication, real-time feedback, and a seamless user experience through the integrated use of Firebase, Firestore, and Stripe.

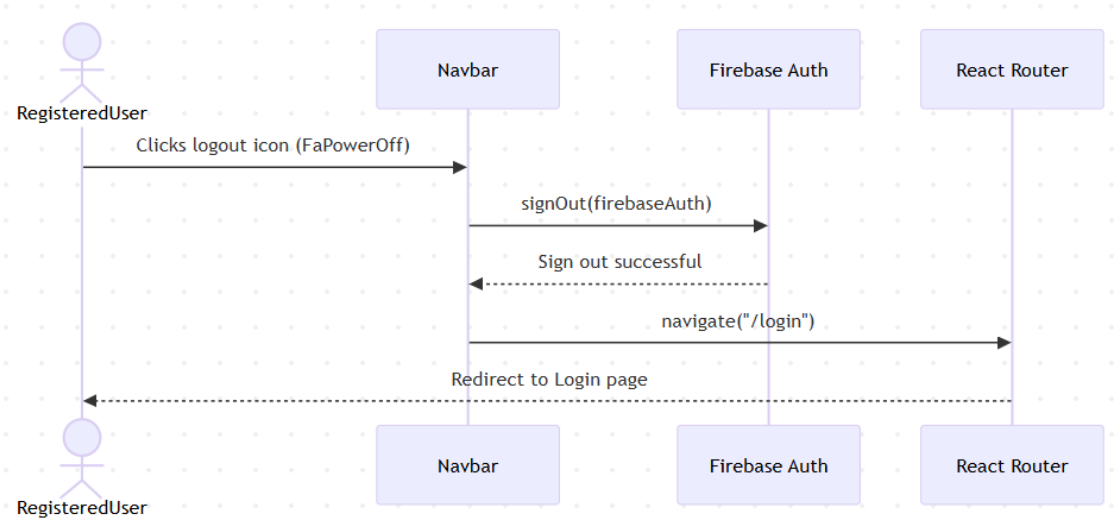

Figure 10: Registered User Logging Out

Figure 10 illustrates When the user clicks the logout icon in the Navbar, it triggers Firebase's signOut() method. Once the authentication session is terminated successfully, React Router then redirects the user to the /login route, completing the logout flow and returning the user to the login page.

6.1.3 SYSTEM PREMIUM USER

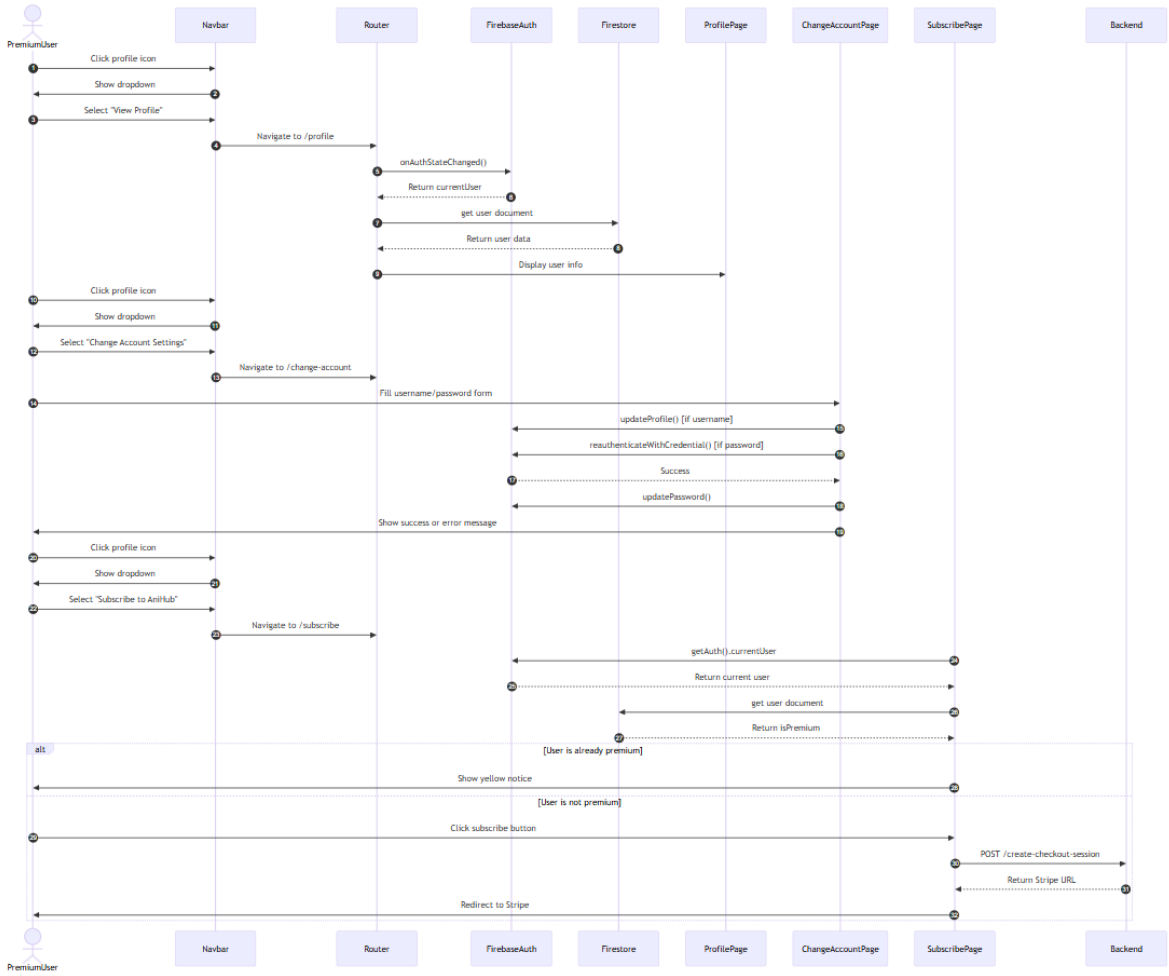

Figure 11: Registered User Account Management and Subscription Navigation

Figure 11 illustrates the navigation and account management flow available to registered users on AniHub, regardless of their subscription status. From the navigation bar, users can access a dropdown menu offering options to view their profile, change account settings, or manage their subscription. Selecting "View Profile" triggers Firebase Authentication to confirm the user's identity and fetches their subscription status from Firestore, displaying relevant account details along with a fallback loading message if needed. Choosing "Change Account Settings" allows the user to update their display name or password. If the password is updated, Firebase requires reauthentication using the current password for security. Finally, selecting "Subscribe to AniHub" checks the user's current premium status. If the user is not yet subscribed, the system initiates a Stripe Checkout session to handle payment. If they are already a premium member, a notice is displayed, but they may still renew their subscription.

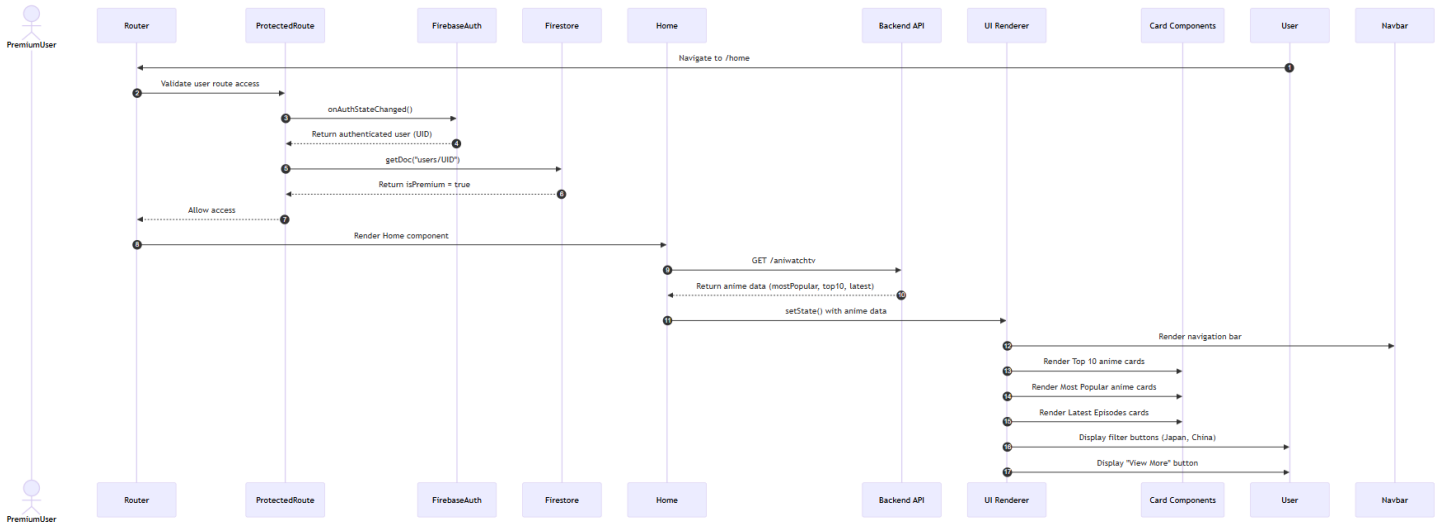

Figure 12: Premium User Homepage

Figure 12 illustrates the complete sequence of events that occurs when a premium AniHub user accesses the home page. Upon navigation, the ProtectedRoute component first verifies the user's authentication status through Firebase Authentication. If the user is logged in, the system fetches their Firestore document to confirm their premium status. Once validated, access to the Home page is granted. The Home component then sends a request to the backend API endpoint aniwatchtv, which returns categorized anime data including Most Popular, Top 10 (Today), and Latest Episodes. This data is stored in React state and triggers a full UI render. The Navbar, filter buttons (for Japanese and Chinese anime), and a “View More” button are displayed, along with dynamically rendered Card components for each anime category. This seamless flow ensures that only premium users receive access to curated anime content while maintaining real-time rendering based on up-to-date data from the backend API.

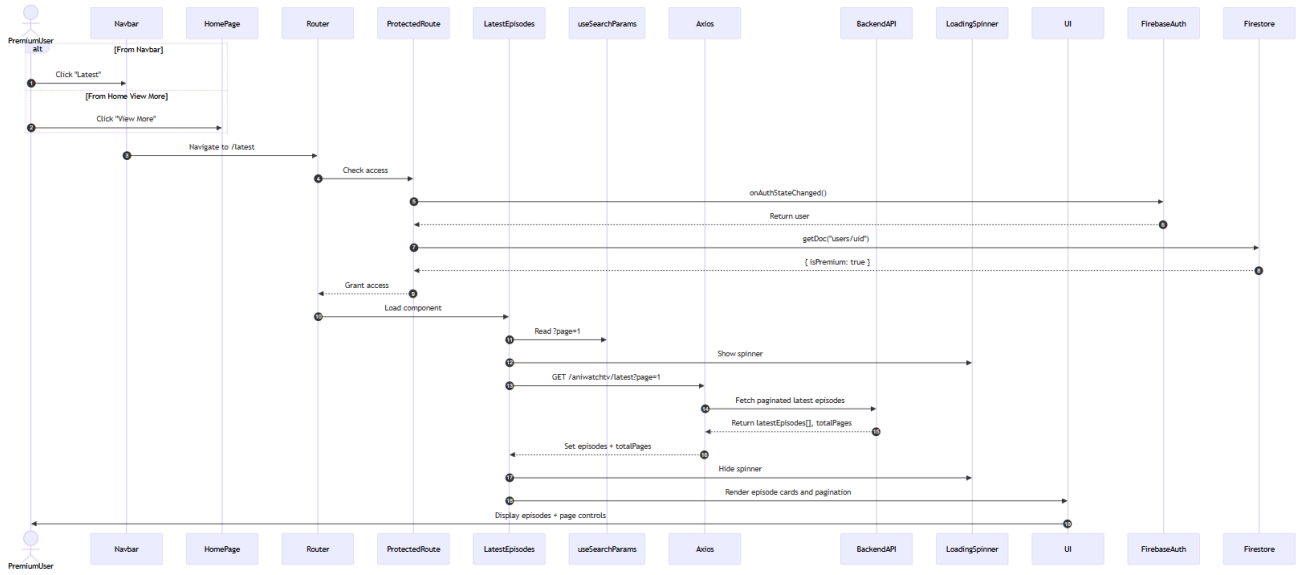

Figure 13: Premium User Anime Origin Filtering and Pagination Flow

Figure 13 illustrates when a Premium User selects an origin filter either *"Japanese"* or *"Chinese"* from the AniHub homepage. This user interaction initiates with a filter button click, which triggers a client-side route change to /origin/:country via the Router. Once navigated, the OriginAnimePage component is rendered and immediately processes the current origin type and page number through parameter extraction (modeled here as ParameterHandler).

Instead of querying the backend, the page loads a locally cached JSON dataset (anime-origin-data.json) using a lightweight LocalDataLoader. Upon successful retrieval, the AnimeFilter module refines the data by filtering titles that match the specified origin and paginates the results into 20-item chunks for better performance and user readability.

Finally, the filtered data is rendered by the UI component, which displays anime cards, the appropriate section heading, and interactive pagination controls. This sequence provides a seamless, responsive browsing experience tailored to regional preferences—without unnecessary API overhead—enhancing overall user satisfaction for premium members.

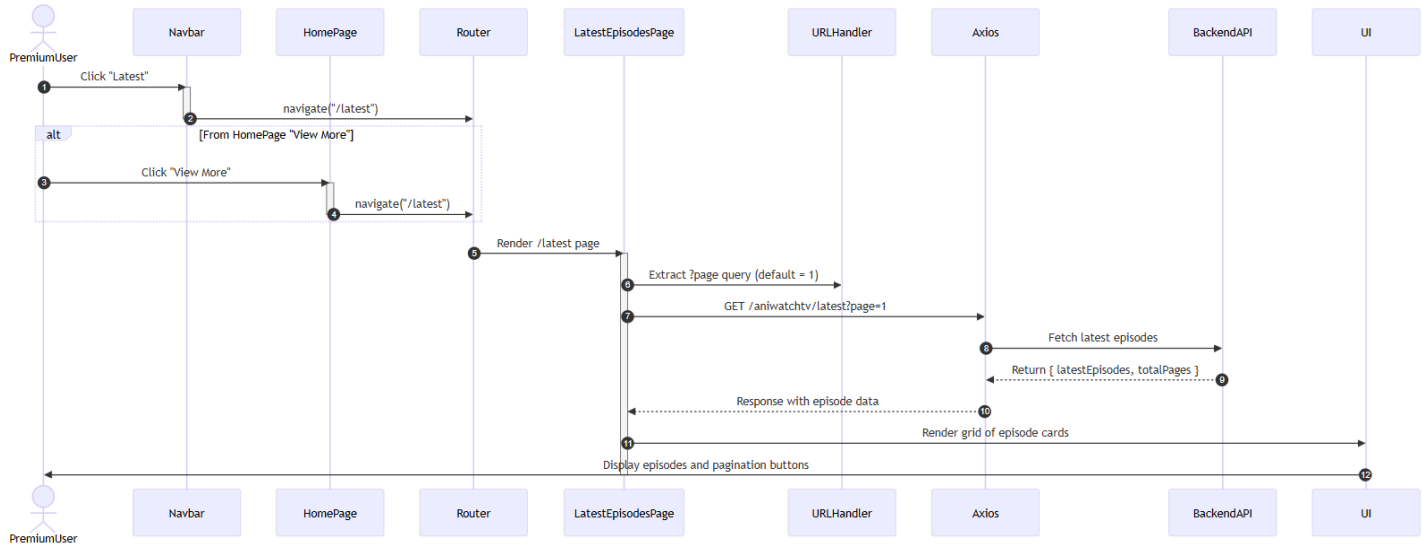

Figure 14: Premium User Viewing Latest Episode

Figure 14 illustrates the sequence when a premium user accesses the latest anime episodes on AniHub, either through the “Latest” link in the navigation bar or the “View More” button within the home page’s episode section. Upon interaction, the system routes the user to a protected page that verifies authentication and premium status using Firebase Authentication and Firestore. Once validated, the page loads and initiates a client-side request to the backend API for the most recent episode data, paginated for performance. While awaiting data, a loading spinner is displayed to enhance user experience. The returned dataset is then rendered as anime cards along with pagination controls, enabling seamless browsing of updated content.

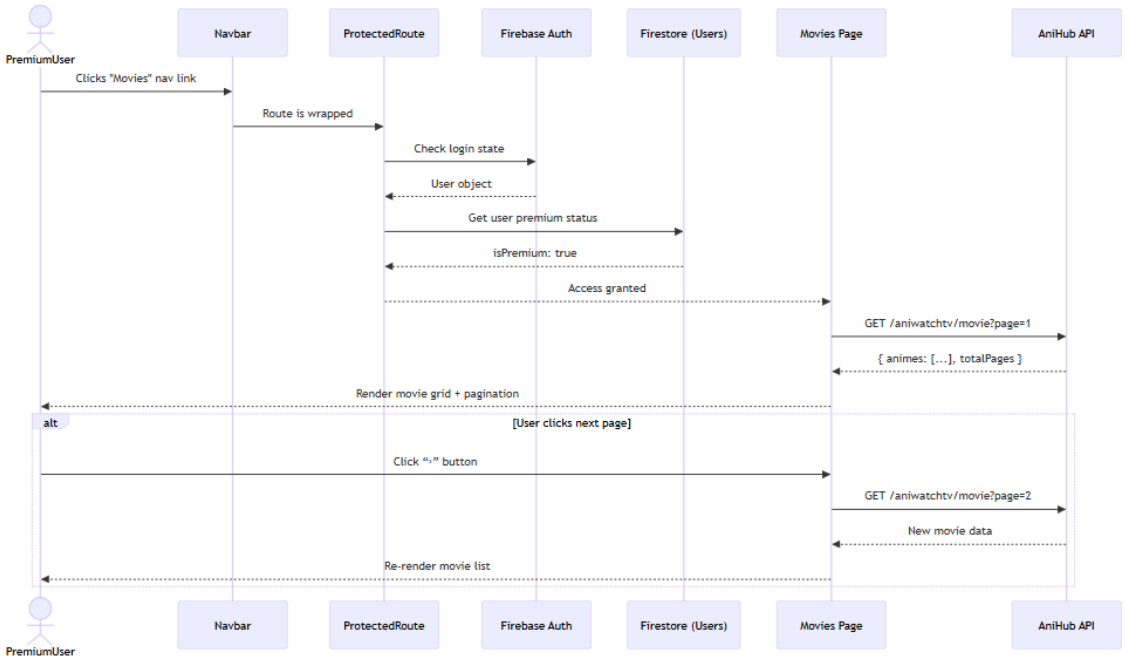

Figure 15: Premium User Viewing Movies

Figure 15 illustrates when a premium user attempts to access the Movies page within the AniHub application. Upon clicking the "Movies" link in the navigation bar, the request is intercepted by the ProtectedRoute component to enforce access control. This component initiates a verification process by first checking the user's authentication state via Firebase Authentication. Once authenticated, the system proceeds to query Firestore to determine whether the user holds premium status by retrieving the isPremium flag from the user's Firestore document. If access is granted, the Movies page is rendered. At this point, the page initiates a request to the AniHub API endpoint /aniwatchtv/movie?page=1 to fetch the first batch of movies. The response includes a list of anime movies along with pagination data, which is then displayed in a dynamic grid layout. Should the user navigate to the next page using pagination controls, the frontend makes another API call to fetch the corresponding data for the next page. This interaction model ensures secure access to premium content, real-time rendering, and a seamless viewing experience for the user.

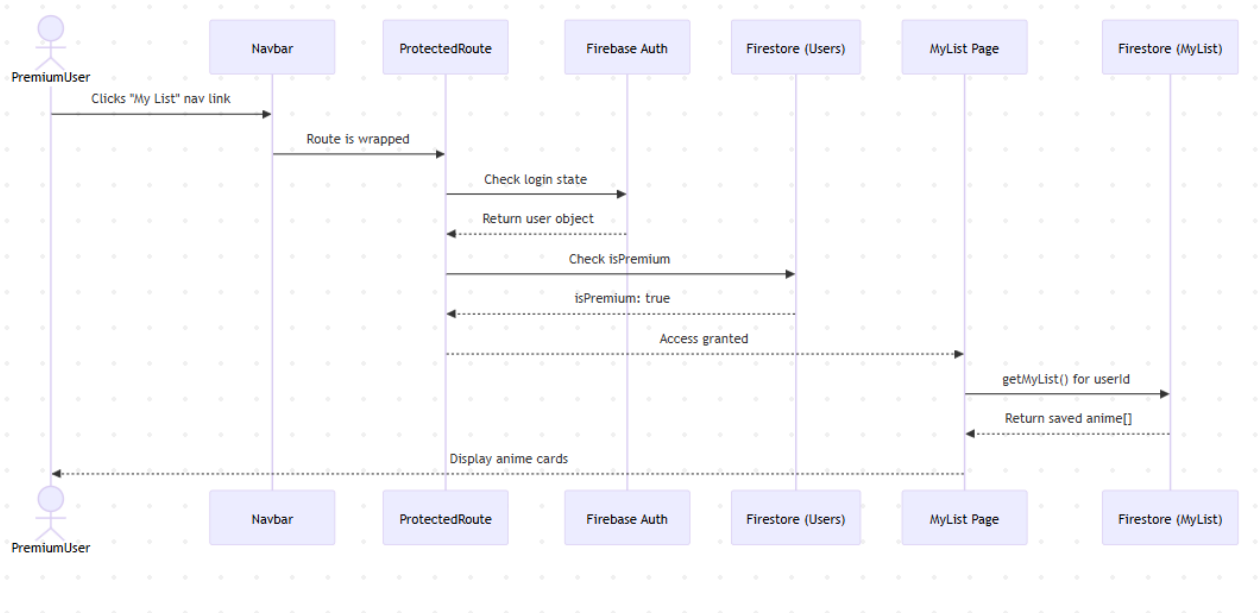

Figure 16: Premium User Viewing Movies

Figure 16 Illustrates when a premium user accesses the "My List" feature in the AniHub application. Upon selecting the "My List" link from the Navbar, the ProtectedRoute component first validates the user's authentication status via Firebase Authentication. It then queries the Firestore database to confirm the user's premium access by checking the isPremium flag in the corresponding user document. Once the user is authorized, the MyList page component renders and invokes a utility function (getMyList) that queries Firestore for the anime titles saved by the current user. The data is retrieved asynchronously and displayed in a styled card grid format. If the list is empty or still loading, the interface provides appropriate feedback to the user. This interaction highlights the seamless integration between Firebase Authentication, Firestore, and the React frontend to securely manage and display personalized, premium-only content.

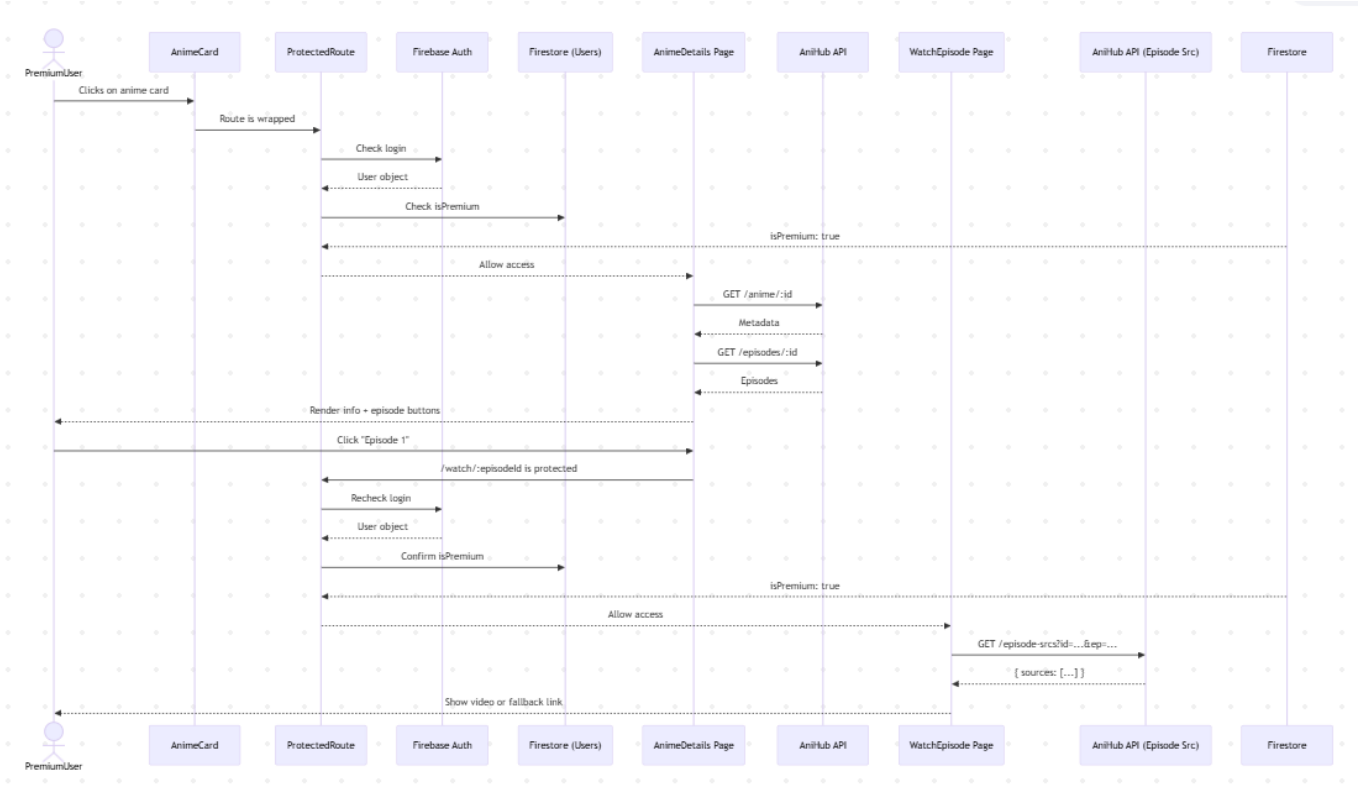

Figure 17: Premium User Viewing and Playing an Anime Episode

Figure 17 illustrates the process a premium user follows to watch an anime episode. The flow begins when the user clicks on an anime card, triggering a route transition that is protected by the ProtectedRoute component. Firebase Authentication checks the user's login status, and Firestore verifies the isPremium flag to confirm subscription.

Upon successful validation, the AnimeDetails page loads and fetches the anime's metadata and episode list from the AniHub API. When the user selects an episode—such as "Episode 1"—another route guard is applied to the /watch/:episodeId path, repeating the authentication and premium check.

If the user is authorized, the WatchEpisode page requests the video source from the backend. Once received, a fallback link is provided for manual playback.

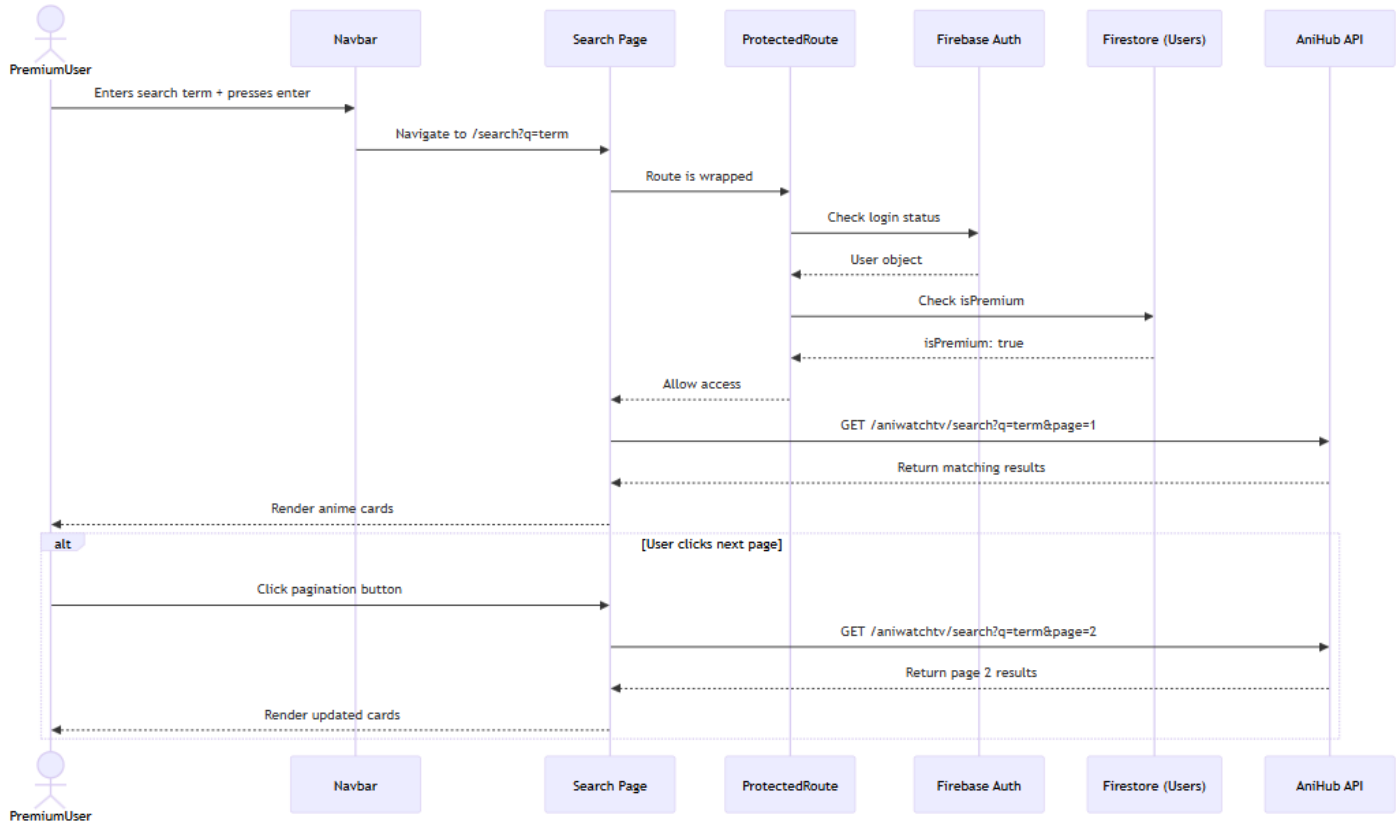

Figure 18: Premium User Searching Anime

Figure 18 illustrates the sequence of events when a premium user performs a search on AniHub. The process begins when the user enters a search term and submits it via the Navbar, which navigates to the /search page with the query attached. This route is wrapped by the ProtectedRoute component, which ensures the user is authenticated and verifies their premium status using Firebase Authentication and Firestore.

Once access is granted, the Search Page sends a GET request to the AniHub API with the search term and retrieves matching anime results. The page then renders these results in a grid format for the user. If the user interacts with the pagination, the system requests the next set of results from the backend and updates the display accordingly.

7.0 INTERFACE DESIGN

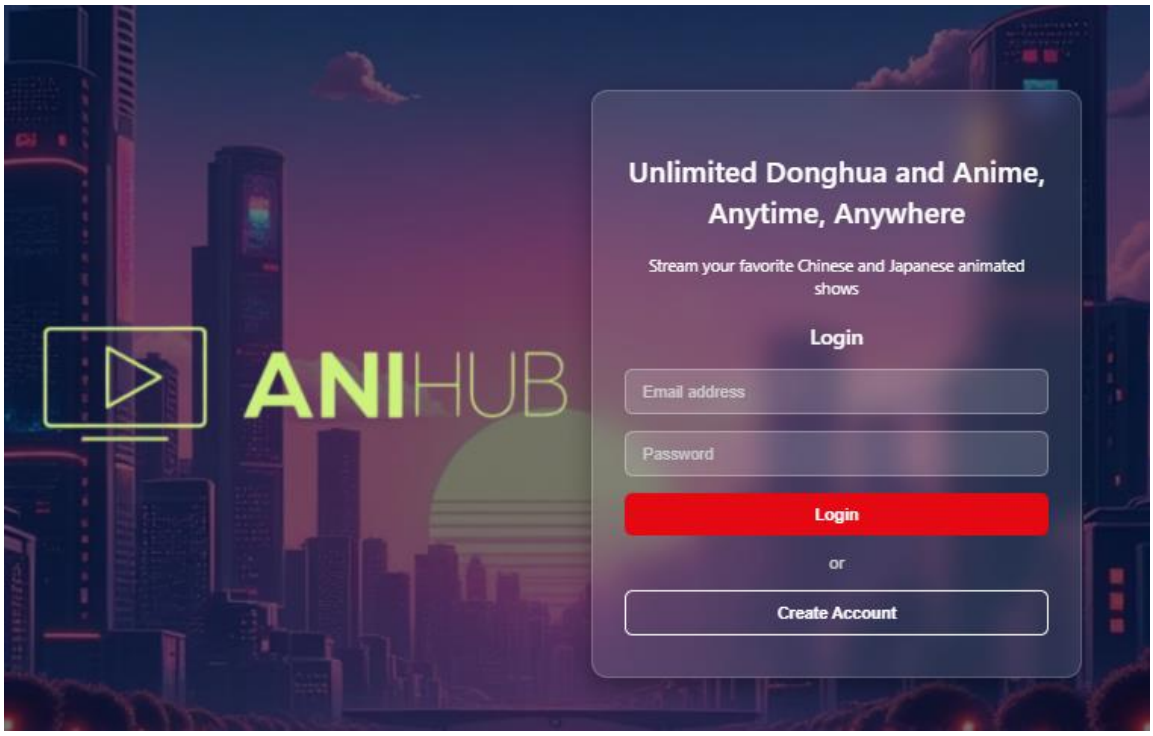

Figure 19: Login Interface

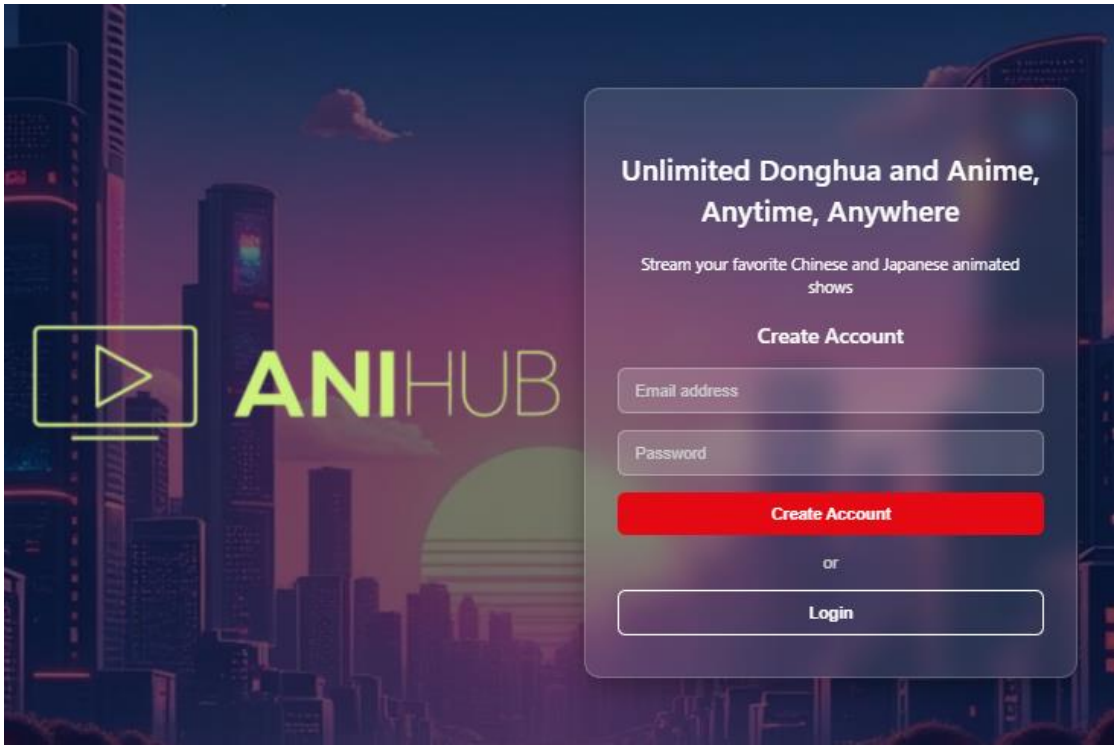

Figure 20: Create Account Interface

8.0 DATABASE

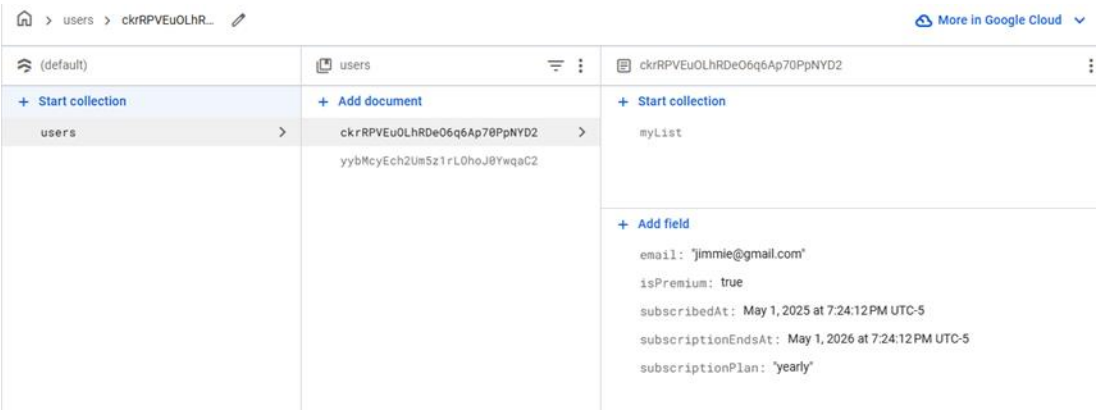

Figure 21: Firebase Firestore Database Schema

The AniHub platform utilizes Firebase Firestore, a cloud-based NoSQL database, to store and manage all user and content-related data in real-time. This database design supports AniHub's core functionalities, including authentication, premium subscription tracking, personalized anime lists, and user activity logging.

Firestore's document-based structure aligns with AniHub's modular architecture. The primary collection is users, where each document represents a registered user identified by their unique Firebase UID. Each user document contains key fields.

Nested within each user document is a myList subcollection. This stores individual anime entries the user has liked or saved for later viewing. Each document in myList represents a saved anime and contains identifiers used for rendering in the frontend.

This structure enables:

- Efficient read/write operations, ideal for real-time streaming apps.
- Scalable and flexible data modeling as user activity grows.
- Secure access control using Firebase rules tied to user authentication.

In addition, Firebase Authentication is used in tandem to manage user sign-in and securely associate user records in Firestore. The integration between Authentication and Firestore ensures a seamless login experience while maintaining strict access control over user-specific data.

This schema was chosen for its speed, simplicity, and seamless Firebase integration allowing AniHub to scale with user demand while maintaining a smooth and responsive UI experience.

9.0 CLASS DIAGRAM

AniHub is built using React, a modern JavaScript library focused on building user interfaces through functional components rather than traditional object-oriented class structures. React encourages the use of hooks such as useState and useEffect to manage state and side effects, allowing for a more modular and declarative approach to development. Because the system architecture relies on functional programming patterns rather than class-based models, a UML class diagram is not applicable in the traditional object-oriented sense. Instead of modeling relationships between classes, the app organizes logic through components, services (such as API calls and Firebase integrations), and state management. For a more accurate representation of the system's behavior, sequence diagrams have been provided to demonstrate how data flows and components interact within the application.

10.0 PROJECT SYSTEM

10.1.1 SYSTEM FUNCTIONALITY

AniHub is a full-stack web-based anime and donghua streaming platform designed to deliver a streamlined, centralized experience for anime enthusiasts. The system integrates a modern frontend, robust backend services, and secure user authentication and subscription mechanisms. Built using React, Firebase, Stripe, Node.js, and TypeScript, AniHub leverages third-party sources such as AniWatchTV to dynamically aggregate and deliver content.

At its core, AniHub provides three levels of user access:

- **Unregistered Users**
  - Can only access the login and signup pages.
- **Registered Users**
  - Can access the homepage but are restricted from premium features.
- **Premium Users (subscribers)**
  - Unlock access to advanced content like full episode playback, movie listings, search, and watchlist features.

The system is divided into the following key functional modules:

Authentication and Authorization:

- Managed via Firebase Authentication.
- Users sign up or log in securely, and their session state determines route access.
- Premium status is stored and verified through Firebase Firestore.

Content Aggregation Module:

- The backend, built with Node.js and Axios, scrapes anime metadata and episode links from AniWatchTV.
- Streaming sources are decrypted using custom parsers.
- Results are categorized into sections such as Most Popular, Top 10, and Latest Episodes.

Subscription Module:

- Premium subscriptions are managed through Stripe.
- A secure backend route updates Firestore when payment is confirmed.
- Premium users are granted full access via route guards.

Routing and Navigation:

- ProtectedRoute components ensure only authenticated and premium users can access certain features like My List, Movies, or Episode playback.
- Routes are dynamically loaded and checked against Firestore premium flags.

Watchlist Management:

- Users can save anime titles to a personalized "My List."
- Data is stored in each user's Firestore document under a dedicated subcollection.

Search and Filter:

- Premium users can search anime by title.
- The origin filter allows users to view anime based on Japanese or Chinese origin.
- These operations rely on pre-fetched data and local filtering logic for fast performance.

Video Playback:

- Playback is initiated on the WatchEpisode page.
- Users are provided with streaming links fetched from MegaCloud.
- Due to CORS restrictions, a fallback URL is offered for manual playback in external players.

Each of these modules works together to form a cohesive, responsive application. From the moment a user registers to when they stream their favorite anime, the system ensures secure access control, dynamic content delivery, and an intuitive UI experience.

The system architecture not only reflects a realistic simulation of commercial anime platforms but also demonstrates how modern full-stack applications can be implemented to support media streaming, subscription monetization, and user personalization at scale.

11.0 SYSTEM REQUIREMENT

11.1.1 SOFTWARE REQUIREMENT

- **Operating System Environment:** Windows 10/11, macOS, or any Linux distribution
- **Front-End Framework:** React.js (v19+) with React DOM and React Router DOM
- **State Management:** Redux Toolkit
- **Styling Tools:** CSS3, Styled-Components
- **Backend & Services:**
  - Firebase Authentication
  - Firebase Firestore (NoSQL Database)
  - Stripe API (for subscription payments)
- **HTTP Client:** Axios
- **Package Manager:** npm (Node.js v18+ recommended)
- **Development Server:** Create React App (via react-scripts)
- **Browser Compatibility:** Chrome, Firefox, Safari, Edge

11.1.2 HARDWARE REQUIREMENT

- **CPU:** Intel/AMD dual-core processor or higher
- **RAM:** Minimum 4 GB (8 GB recommended for development)
- **Hard Disk:** At least 10GB for dependencies and project files
- **Internet Connection:** Required for Firebase services, API communication, and streaming content

12.0 SYSTEM IMPLEMENTATION

12.1.1 SUBSCRIPTION PLAN SELECTION PAGE

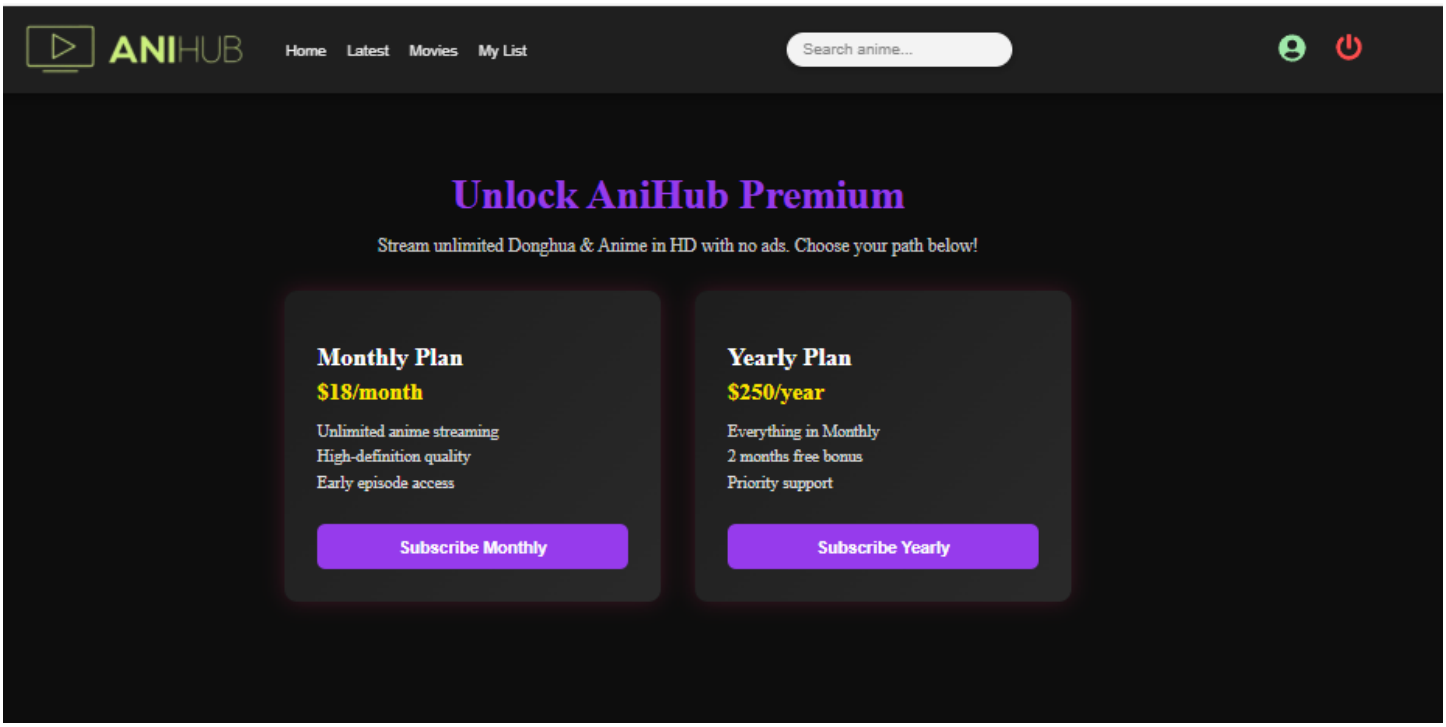

Figure 22: Premium Subscription Page

After a user registers and logs in to AniHub, they are directed to the Premium Subscription page, where they can choose between two subscription tiers: Monthly ($18/month) or Yearly ($250/year). This page is implemented using React with Firebase Authentication and Firestore to determine if the user is a free or premium member. If the user has not yet subscribed, this component presents subscription options via Stripe integration. Upon selecting a plan, the user is redirected to Stripe Checkout, and upon successful payment, their isPremium status is updated in Firestore, unlocking access to premium features like the Movies page, My List, and episode streaming.

12.1.2 AUTHENTICATED HOME DASHBOARD

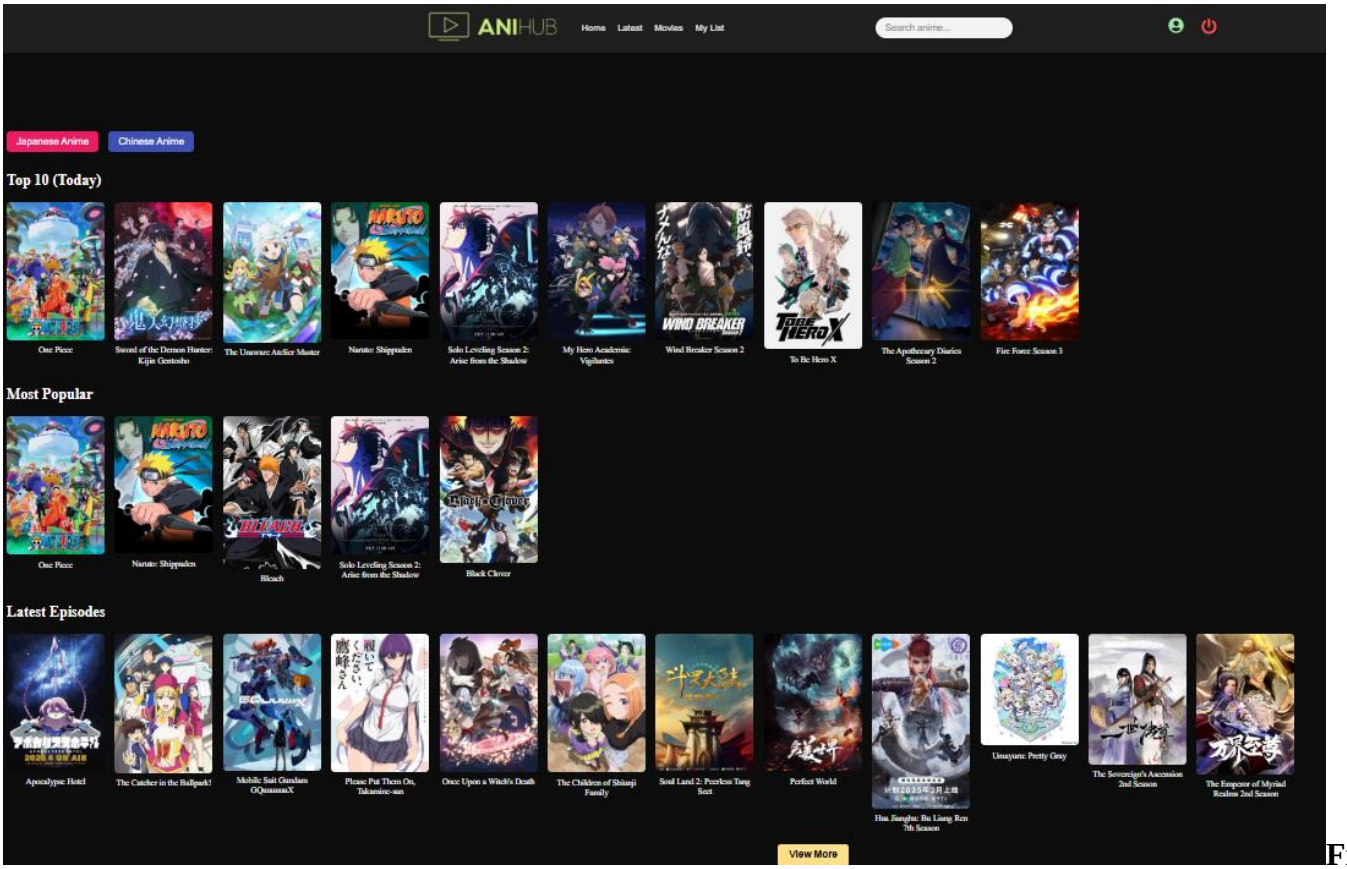

Figure 23: Authenticated Home Dashboard

The Home interface of AniHub is the main landing page for logged-in users, designed with a focus on usability and visual appeal. It features a sleek dark theme and is built using React with dynamic data fetched from the AniWatchTV API. At the top, users are greeted with the AniHub logo, a fixed navigation bar (Home, Latest, Movies, My List), a search bar, and user profile/logout icons for managing sessions.

Beneath the navigation, the page displays categorized anime sections like Top 10 (Today), Most Popular, and Latest Episodes, each populated with horizontal scrollable anime posters. Each card represents an anime entry, and clicking it navigates users to the AnimeDetails page. Users can also filter anime by Japanese or Chinese origin using two toggle buttons at the top.

The "Latest Episodes" section includes a “View More” button that directs users to a paginated page showing all recently updated episodes. All content visibility is dynamically controlled based on user type (premium or free), with premium users having broader access to categories like Movies and streaming content.

This interface serves as the user's dashboard, combining browsing, filtering, and navigation in a clean, responsive layout.

12.1.3 LATEST EPISODE UPDATE PAGE

Figure 24: Latest Episode Interface

The Latest Episodes Page serves as a centralized hub for browsing the most recently updated anime episodes on AniHub. This page is built using React and dynamically fetches paginated episode data from the backend API connected to AniWatchTV. At the top, the interface maintains the familiar AniHub navbar with navigation links (Home, Latest, Movies, My List), a search bar, and profile/logout controls.

The main content area features a clean grid layout displaying anime cards that represent the newest episodes. Each card includes a poster image and the anime's title, ensuring high readability and visual clarity. The content is scrollable, responsive, and optimized for both desktop and tablet viewing.

At the bottom of the page, a pagination control allows users to browse additional pages of updated episodes. This ensures performance efficiency while handling large datasets and makes content discovery more user-friendly.

This page is accessible only to authenticated users, and the video streaming experience is gated based on premium status. Clicking on an anime redirects the user to the AnimeDetails page, where full episode listings and streaming options are available.

12.1.4 PREMIUM MOVIE PAGE

Figure 25: Premium Movie Page

This page showcases all available anime movies exclusively accessible to premium users. Upon login, users with an active subscription can browse high-quality movie titles, view cover art, and navigate to individual movie detail pages. The content is dynamically fetched from the AniHub API with pagination support, and access is gated using Firebase Authentication and Firestore to ensure only users with isPremium: true can view this section.

12.1.5 ANIME CARD HOVER INTERACTION

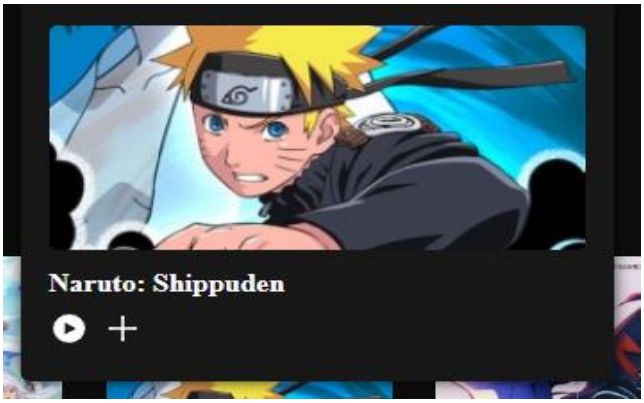

Figure 26: Card Hover Interface

Each anime in AniHub is presented as an interactive Card Hover Component, designed using React and styled-components. When hovered, the card reveals actionable icons such as the Play button (to start watching) and a Plus button (to add the anime to the user's “My List”). These interactions are connected to backend logic through Firebase Firestore, allowing premium users to build and manage their personal watchlists dynamically.

12.1.6 MY LIST PAGE

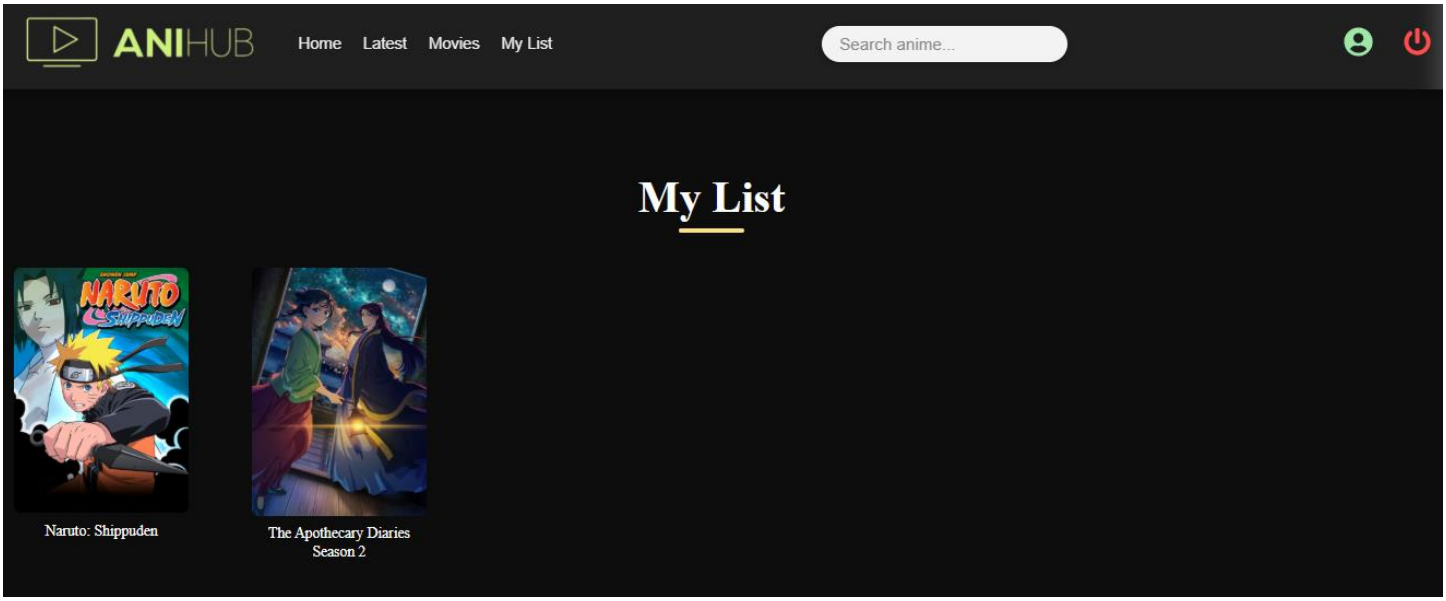

Figure 27: My List Page

The My List page provides authenticated premium users with a personalized space to manage their saved anime. Whenever a user clicks the Plus button on an anime card, that anime is added to their Firestore-stored myList collection. This interface is dynamically populated by fetching the user’s saved anime data on component mount, ensuring a smooth and responsive experience. It enables users to quickly access favorite shows and continue watching at their convenience.

12.1.7 USER PROFILE MANAGEMENT

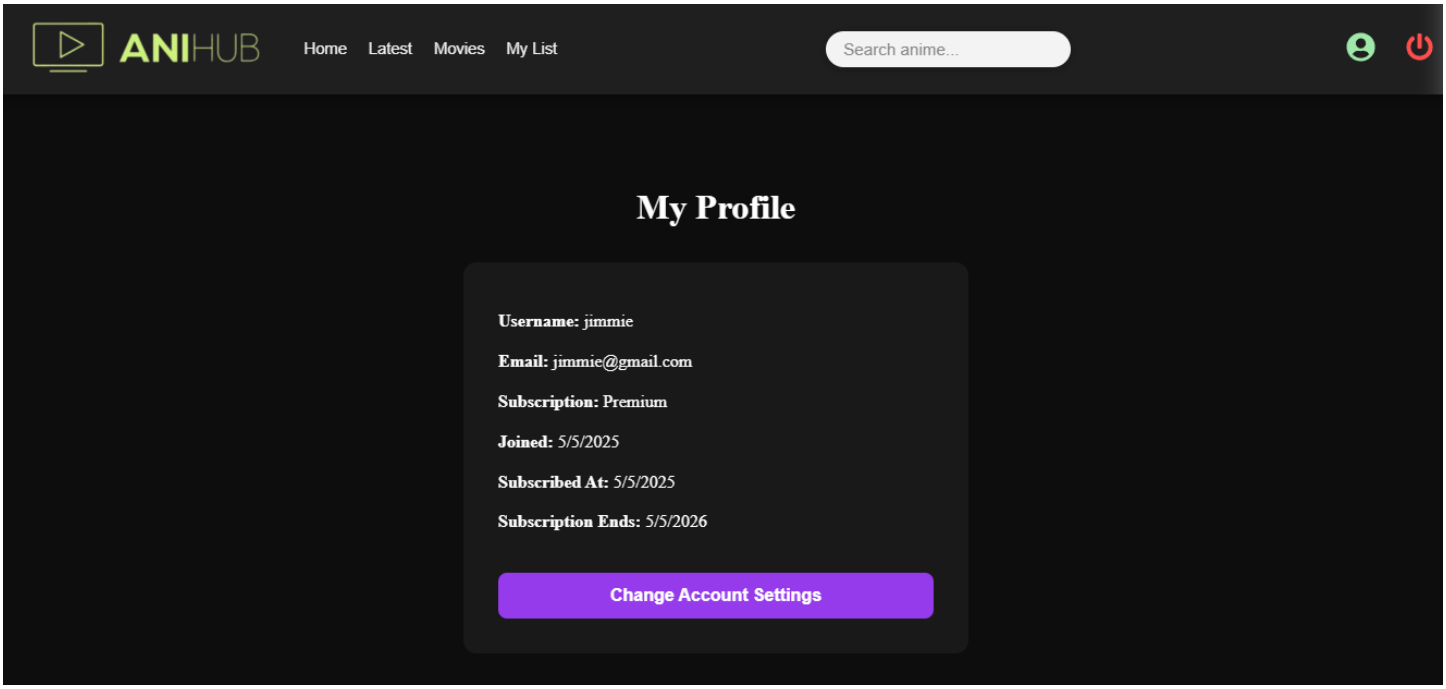

Figure 28: User Profile Management

The User Profile Management interface allows logged-in users to view personal account details including their username, email, subscription type (e.g., Premium), and subscription timeline. This information is fetched directly from Firebase Firestore, providing a centralized, secure source of truth for user identity and billing status. Users are also given the option to modify account settings through a dedicated Change Account Settings button, enhancing control over their profile and subscription lifecycle.

12.1.8 ACCOUNT SETTING & CREDENTIAL UPDATE

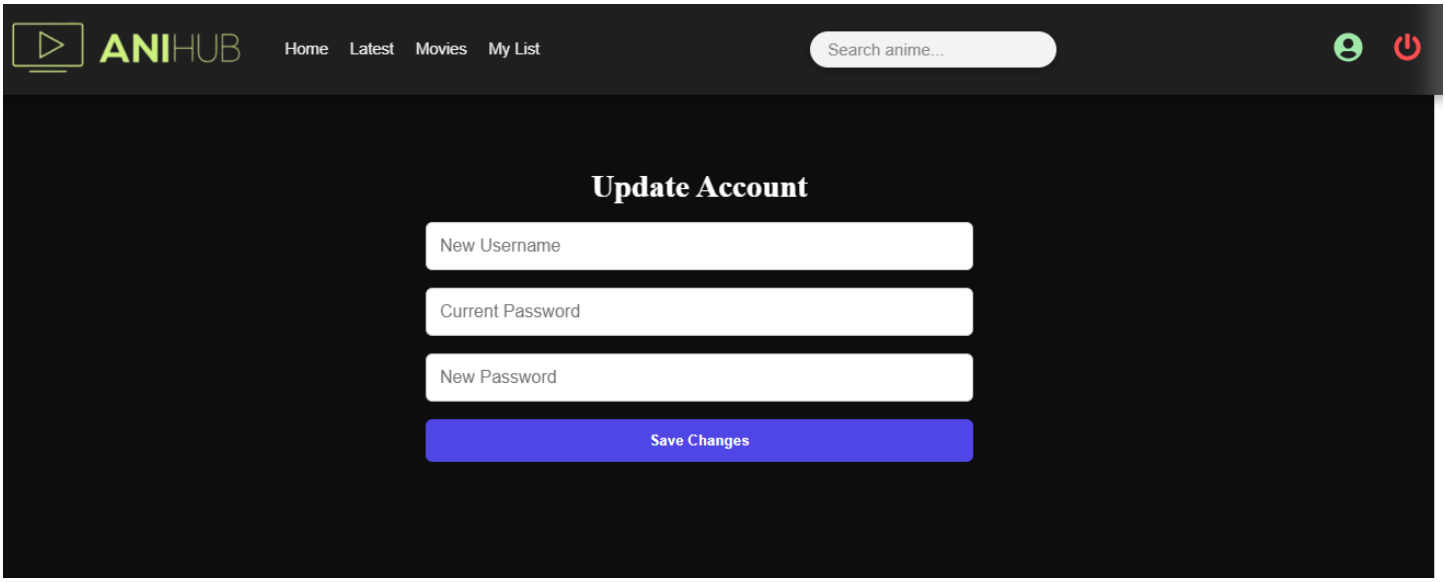

Figure 29: Account Setting & Credential Update

This interface allows authenticated users to update their account credentials securely. Users can change their username and reset their password by providing their current credentials. This feature uses Firebase Authentication to validate and apply updates in real time. It enhances user security and gives control over personal identity within the AniHub platform.

12.1.9 ANIME DETAIL PAGE

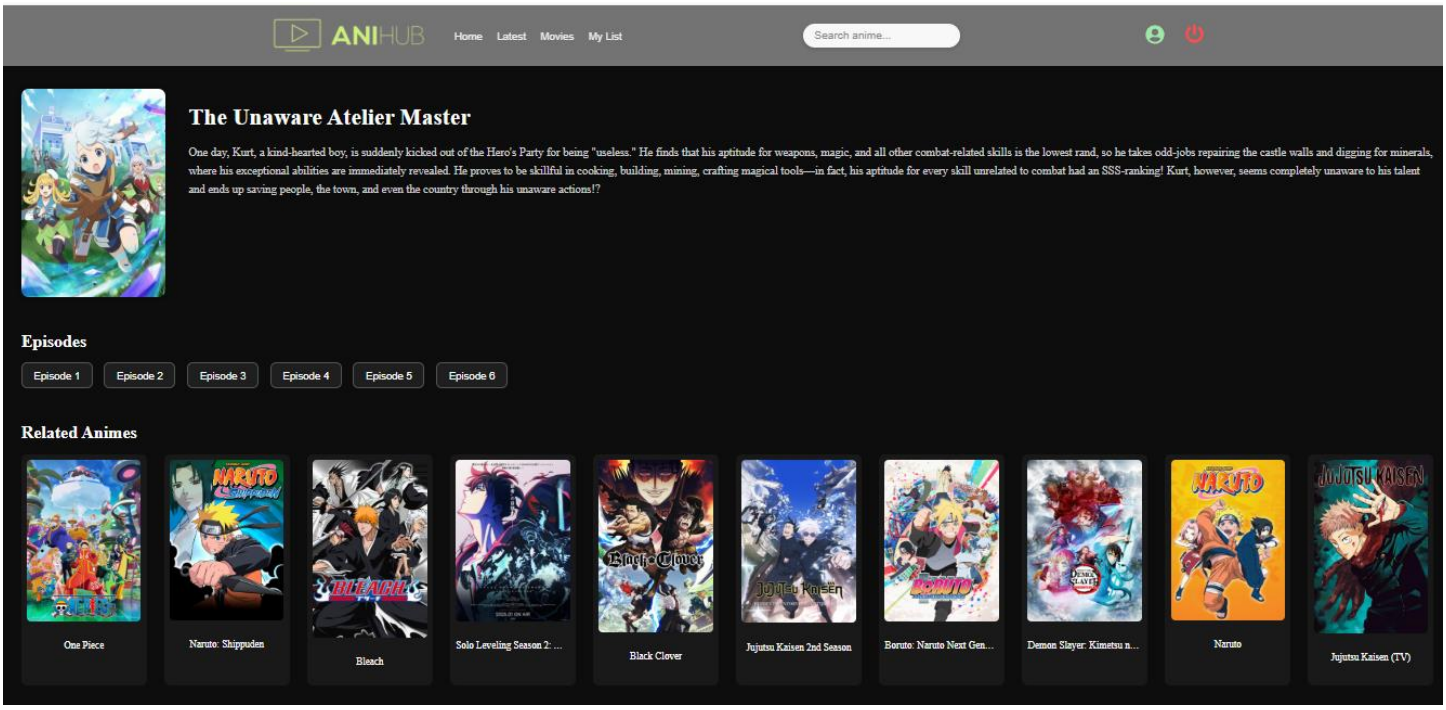

Figure 30: Anime Detail Page

This page displays detailed information about a selected anime, including its synopsis, available episodes, and related anime recommendations. Upon selection, a user is navigated to this route (e.g., /anime/:animeId), where the frontend fetches metadata and episode data from the AniHub API. Episodes are listed as interactive buttons, allowing users to begin playback if subscribed. Related animes are displayed below to encourage continued engagement and discovery.

12.2.0 EPISODE PLAYBACK PAGE

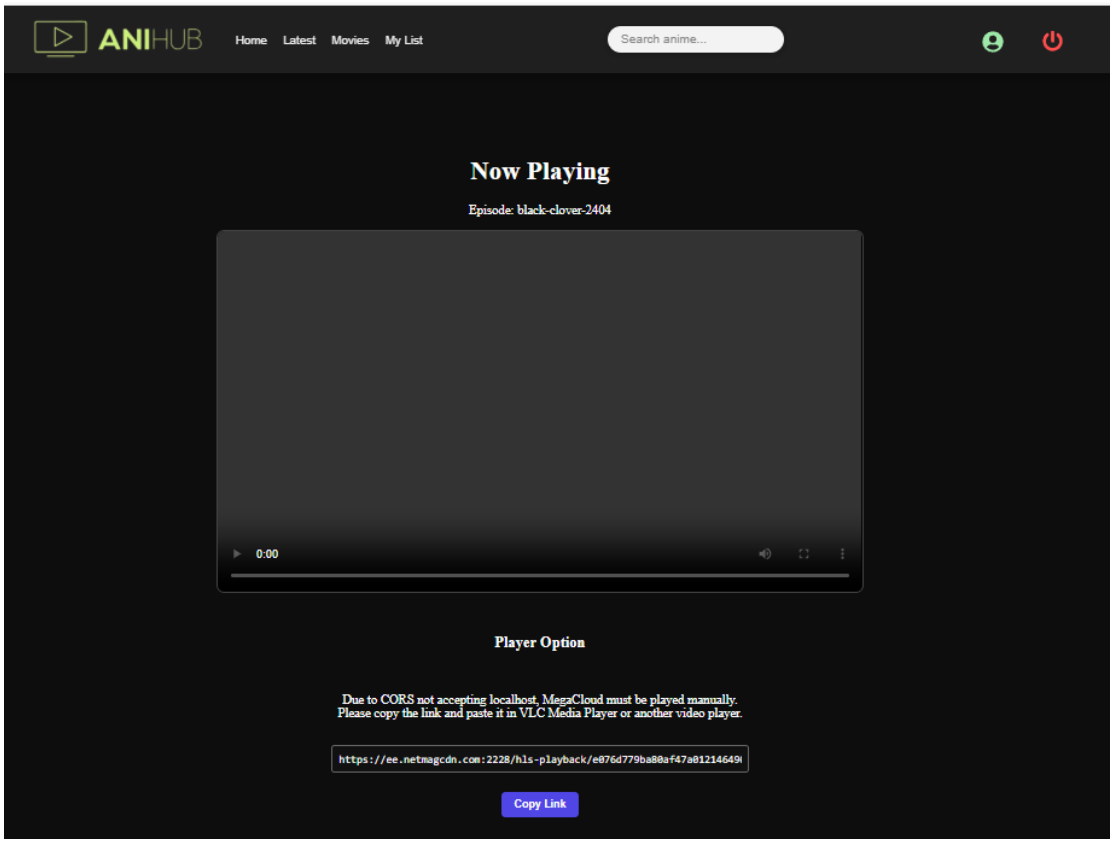

Figure 31: Episode Playback Page

This component is responsible for displaying the selected anime episode. When a user clicks on an episode from the Anime Details page, they are directed to this route (e.g., /watch/:episodeId). The system fetches a streaming source from the backend API. If an HLS stream is available and playable in-browser, the video is displayed. However, due to CORS restrictions with MegaCloud, users are provided with a fallback manual link that can be copied and played using external players like VLC.

13.0 SYSTEM MODIFICATION

13.1.1 SYSTEM USER MODULE

During the development of AniHub, several system modifications were made to better align with evolving project goals and technical requirements. Initially, the system was intended to be a basic content listing site, but it was enhanced to include user authentication using Firebase Authentication, allowing personalized access and subscription-based content gating. Firestore was integrated to store user data, such as premium status and saved anime lists, replacing the need for a traditional relational database. The API layer, originally designed to scrape and serve static anime metadata, was expanded to support pagination, search functionality, and secure streaming URLs from third-party sources like AniWatchTV. The frontend architecture also evolved React components were modularized, and route protection was implemented to ensure that only authenticated premium users could access certain pages like /movies, /mylist, and episode streaming. These modifications significantly improved the scalability, user experience, and maintainability of the AniHub platform.

14.0 SYSTEM LIMITATION

- AniHub cannot host or stream original video content directly. The platform relies on scraping and embedding video sources from third-party sites like AniWatchTV. This creates a dependency on those external services for video availability, playback, and stability.

- The system does not support native video playback for some sources due to CORS restrictions. For example, MegaCloud-hosted videos may not load directly in the browser. As a workaround, users must copy the episode URL and manually paste it into an external video player like VLC.

- AniHub cannot guarantee long-term accuracy of anime data. Since the system scrapes metadata and episode information from a live third-party website, any structural or content changes to that source may cause disruptions or inaccuracies in AniHub's listings.

- Offline access and download functionality are not available. Users must stream episodes online, and no caching or download mechanism is built into the system.

- There are no user-driven social features. The current system lacks commenting, user reviews, or community interaction tools, which may limit user engagement and retention.

- Responsiveness across devices may vary. While AniHub is primarily optimized for desktop experiences, some UI components may not behave ideally on mobile devices or small screen resolutions.

15.0 SYSTEM FUTURE ENHANCEMENT

AniHub has been built with scalability and modularity in mind, allowing for future enhancements that can extend its functionality and reach. In upcoming iterations, the platform could introduce dedicated mobile applications for both Android and iOS devices to provide a seamless streaming experience on phones and tablets, beyond the web interface.

Other possible enhancements include implementing user profiles with personalized recommendations, watch history tracking, and advanced filtering by genre. Social features such as user reviews, ratings, and community discussions can be integrated to boost user engagement.

On the backend, the system may also evolve to support real-time data updates, improve video caching strategies, and reduce dependency on third-party scraping by partnering with official content providers. These improvements will ensure AniHub remains competitive, adaptable, and optimized for a growing user base and expanding anime content library.

16.0 PROJECT SCHEDULING

16.1.1 SCHEDULING TASKS DURATION

TASKS	**DURATION**
Use case	Create use cases, requirements and understanding expectations.
Project proposals	Updates to project description. Updates to mission, scope, requirements. First demo diagrams
Diagrams	Create class, activities, sequence.

Table 1.0 Tasks duration

16.1.2 PROJECT TIMELINE

- **Planning and Requirements Gathering:**

The project began with identifying the goals of AniHub, outlining major features like anime browsing, user authentication, premium subscription handling, and episode streaming.

- **System Design:**

Architecture decisions were made to use React for the frontend and Firebase (Authentication & Firestore) for backend services. A custom Node.js and Express API was set up to scrape anime data from AniWatchTV.

- **Development and Testing:**

Frontend development involved building out modular components such as Login, Signup, Navbar, Home, AnimeDetails, and WatchEpisode. Backend API endpoints were tested to ensure proper pagination, search, and video source handling. Firebase authentication and route protection were tested across various user states (guest, logged-in, premium).

- **End User Documentation:**

Documentation included clear summaries of all routes and user flows. While a class diagram was not applicable due to the use of React's functional component architecture, the project included sequence diagrams.

- **Final Handover:**

The project was finalized with a complete deployment-ready codebase, submission of the thesis report, and a demonstration-ready version of AniHub including user login, subscription flow, and video playback all protected under the premium route guard system.

17.0 CONCLUSION

The primary goal of this project was to design and implement AniHub, a modern anime streaming web application that allows users to browse, search, and stream anime content from third-party sources with a clean interface and premium feature access. Built using React for the frontend and Firebase for backend authentication and data storage, the system focused on delivering a scalable, secure, and user-friendly streaming experience.

Throughout the development process, several challenges were encountered, including CORS restrictions with MegaCloud video sources, API integration with scraped third-party platforms, and implementing secure route protection for premium users. These issues were addressed through architectural strategies such as React ProtectedRoute components, Firestore-based user role validation, and backend enhancements to support decrypted streaming links.

One of the most valuable learning outcomes was understanding how real-world anime streaming websites worked, and similar platforms operate behind the scenes. This included analyzing their content delivery mechanisms, video embedding patterns, and how they organize and structure anime data. These insights greatly informed how AniHub's backend scrapers and frontend rendering logic were designed and implemented.

In summary, AniHub successfully meets its objectives within the time given as a functional streaming platform and lays a solid foundation for future enhancements. The knowledge gained throughout this project reflects both technical growth and real-world application of web technologies in the anime streaming domain.

18.0 REFERENCES

ONLINE ARTICLES

**AUTHOR, (YEAR OF PUBLICATION). TITLE, VOLUMN NUMBER (ISSUE NUMBER IF ANY). RETRIEVED MONTH DAY, YEAR FROM URL

Falcon71181. (n.d.). *Anime API – AniWatchTV Scraper and Streaming Backend*. GitHub. Retrieved May 1, 2025, from https://github.com/falcon71181/Anime-API?tab=readme-ov-file#aniwatch